Platzhalter

Lo
go

Hausarbeit

Beispieltitel
für einen Bericht

Autor:	Vorname Nachname
Matrikel-Nr.:	xxxxxxx
Studiengang:	Maschinenbau
Erstprüfer:	Prof. Dr. Elmar Wings
Abgabedatum:	5. Mai 2026

Hochschule Emden/Leer · Fachbereich Technik · Abteilung Maschinenbau
Constantiaplatz 4 · 26723 Emden · http://www.hs-emden-leer.de

Inhaltsverzeichnis

Inhaltsverzeichnis 3

Abbildungsverzeichnis 5

Liste des Listings 7

1. Introduction 11

I. Domain Knowledge - Application 13

2. Application 15

II. Domain Knowledge - Hardware 17

3. Hardware XY 19
3.1. Further Readings . . . 19

4. Ultraschallsensor HC-SR04 21
4.1. Allgemeines . . . 21
4.2. Ultraschallsensor HC-SR04 . . . 21
4.3. Spezifikation . . . 21
4.4. Bibliothek . . . 22
4.4.1. Beschreibung . . . 22
4.4.2. Installation . . . 22
4.4.3. Funktionen . . . 22
4.5. Beispiel . . . 23
4.5.1. Handbuch . . . 23
4.5.2. Funktion vom Beispiel . . . 24
4.5.3. Code . . . 24
4.6. Kalibrierung . . . 25
4.7. Einfacher Code ohne newPing Bibliothek . . . 25
4.8. Einfache Anwendung . . . 26
4.9. Tests . . . 27
4.9.1. Einfacher Funktionstest . . . 27
4.9.2. Alle Funktionen testen . . . 27
4.10. Weiterführende Literatur . . . 28

5. SG90 Servomotor 29
5.1. Allgemeines . . . 29
5.2. Servomotor TowerPro SG90 . . . 29
5.3. Spezifikation . . . 29
5.4. Bibliothek . . . 30
5.4.1. Beschreibung . . . 30
5.4.2. Installation . . . 30
5.4.3. Funktionen . . . 30

5.5. Beispiel . . . 31
5.5.1. Handbuch . . . 31
5.5.2. An einem Beispiel . . . 31
5.5.3. Code . . . 31
5.5.4. Dateien . . . 32
5.6. Kalibrierung . . . 32
5.7. Einfacher Code (ohne Bibliothek) . . . 32
5.8. Einfache Anwendung . . . 33
5.9. Tests . . . 34
5.9.1. Einfacher Funktionstest . . . 34
5.9.2. Alle Funktionen testen . . . 34
5.10. Weiterführende Literatur . . . 34

Literatur **35**

III. Domain Knowledge - Tools **37**

6. Python Tool XY **39**

7. Hardware Tool XY **41**

IV. Developmenmt **43**

8. Flow Chart Development XY **45**

V. Monitoring **47**

9. Monitoring XY **49**

VI. Verification - Evaluation - Conclusion **51**

10. Evaluation/Verification XY **53**

11. To DoX Y **55**

12. Conclusion XY **57**

VII. Anhang **59**

13. Bill of Material **61**

14. SBOM **63**

15. Package `Example` **65**
15.1. Introduction . . . 65
15.2. Description . . . 65
15.3. Installation . . . 65
15.4. Example - Manual . . . 65
15.5. Example . . . 65
15.6. Example - Code . . . 65
15.7. Example - Files . . . 65

15.8. Further Reading 65

VIII Hints - Examples for LaTeX - Read, Use and Remove 67

16. Hinweise 69
16.1. Allgemeine mathematische Beschreibung Bézier-Kurve 70

17. Verrundung mit einem Kreisbogen 73
17.1. Gleichungen 73
17.2. Bewertung 75
17.3. Beispiele 77

18. Maple-Dateien 81
18.1. Geometrien mit Maple 81
18.2. Verwendete Geometrieelemente 81
18.3. Aufbau der Datenstruktur 82
18.4. Struktur eines Moduls 82
18.4.1. Genereller Aufbau eines Moduls 84
18.4.2. Sicherung eines Moduls 86
18.4.3. Verwendung eines Moduls 86
18.4.4. Erstellung eines Maple-Moduls 86
18.5. Funktionen der Module 87
18.5.1. Modul `MPoint` 87
18.5.2. Modul `MLine` 88
18.5.3. Modul `MArc` 89
18.5.4. Modul `MBezier` 89
18.5.5. Modul `MPolygon` 90
18.5.6. Modul `MGeoList` 90
18.5.7. Modul `MHermiteProblem` 91
18.5.8. Modul `MHermiteProblemSym` 91
18.5.9. Modul `MConstant` 92
18.5.10. Modul `MGeneralMath` 92
18.5.11. Modul `Biarc` 93
18.5.12. Modul `MBiarc` 93
18.6. Programmablaufplan 94
18.6.1. Gesamauf 94
18.6.2. Verrundung der Kurve 94

19. Representation of Python Programs 97

20. Representation of Programs written for Arduino Boards 99

21. Erstes Kapitel 101

22. CAGD 103

A. Zeichnungen mit tikz 105

B. Kriterien für eine gutes LaTeX-Projekt 107

Literaturverzeichnis 109

Index 111

Abbildungsverzeichnis

4.1. Schematische Darstellung des Ultraschallsensors HC-SR04 22

16.1. Bernstein-Polynom vom Grad 3 71
16.2. Bézier-Kurve zum Beispiel 16.1 72

17.1. Glättung einer Ecke mit Hilfe eines Kreisbogens - Dreieck 73
17.2. Winkeländerung bei einer Verrundung mit einem Kreisbogen 75
17.3. Krümmungsverlauf bei der Verrundung mit einem Kreisbogen 76
17.4. Maximaler Bereich für die Verrundung mit einem Kreisbogen - $L(\varepsilon = const, \alpha)$ 76
17.5. Abweichung bei Vorgabe des Abstands L bei der Verrundung mit einem Kreisbogen 76

18.1. Programmablaufplan „Eckenverrrundung“ 95
18.2. Programmablaufplan „Auswal der Verrundungsstrategien“ 96

Liste des Listings

4.1. Beispiel Code 24
4.2. Beispiel Code ohne Bibliothekseinbindung 25
4.3. Einparkhilfe Beispielcode 26

5.1. Beispiel Code für den TowerPro SG90 31
5.2. Code ohne Bibilothek für den TowerPro SG90 32

19.1. „Hello World“ in Python – Variant 1 98

20.1. „Hello World“ in Python – Variant 1 100

Abkürzungen

CNC Computerized Numerical Control

SPS Speicherprogrammierbare Steuerung

1. Introduction

Flettner rotors are now widely used [1,2,3]. ...

The construction of a Flettner rotor for a water taxi [4] ...

External influences pose great challenges to [5,6] ...

Through the use of 3D printed parts, ...

In the second chapter, ...

Teil I.

Domain Knowledge - Application

2. Application

Teil II.

Domain Knowledge - Hardware

3. Hardware XY

3.1. Further Readings

4. Ultraschallsensor HC-SR04

Einleitung

Dieses Kapitel befasst sich mit der berührungslosen Abstandsmessung mithilfe von Sensoren. Zunächst wird ein allgemeiner Überblick über Ultraschallsensoren zur Distanzmessung gegeben. Anschließend wird der in diesem Projekt verwendete Ultraschallsensor HC-SR04 detailliert beschrieben, einschlieSSlich seiner technischen Spezifikationen und der softwareseitigen Implementierung unter Verwendung ausgewählter C++ Bibliotheken.

4.1. Allgemeines

Sensoren zur Abstandsmessung sind wichtige Komponenten in der modernen Automatisierungstechnik, Robotik und Fahrzeugtechnik [Fra16]. Sie ermöglichen es Systemen, eine Umgebung räumlich zu erfassen. Ultraschallsensoren arbeiten nach dem Echo-Lot-Prinzip (Schalllaufzeitmessung). Sie senden Schallwellen aus und messen die Zeit, bis das Echo zurückkehrt [Ele20]. Sie sind unabhängig von der Farbe oder optischen Transparenz eines Objekts, haben jedoch ihre Grenze bei schallabsorbierenden Materialien (z. B. Schaumstoff) [Fra16].

4.2. Ultraschallsensor HC-SR04

In diesem Projekt kommt der Ultraschallsensor **HC-SR04** zum Einsatz. Er wandelt die gemessene Entfernung in ein PWM-Signal um, welches am Ausgang zur Verfügung steht [KT-12]. Das Auslösen eines Messzyklus geschieht durch eine fallende Flanke am TRIG-Eingang für mindestens 10 µs. Das Modul sendet darauf nach ca. 250 µs ein 40 kHz Burst-Signal für die Dauer von 200 µs aus. Danach geht der ECHO-Ausgang sofort auf HIGH-Pegel und das Modul wartet auf den Empfang des Echos. Wird dieses detektiert, fällt der Ausgang auf LOW-Pegel. Wird kein Echo detektiert, verweilt der Ausgang für insgesamt 200 ms auf HIGH-Pegel und zeigt so eine erfolglose Messung an. Ein vollständiges Messintervall hat eine Dauer von 20 ms, sodass maximal 50 Messungen pro Sekunde durchgeführt werden können [KT-12].

4.3. Spezifikation

Die elektrischen und physikalischen Parameter entstammen den offiziellen Spezifikationen von KT-elektronic [KT-12].

- Betriebsspannung (VCC: +5 V DC ±10%)
- Stromaufnahme: < 2 mA
- Messbare Distanz: 2 cm bis ca. 300 cm
- Messauflösung: 3 mm
- Messintervall: 20 ms (maximal 50 Messungen pro Sekunde)

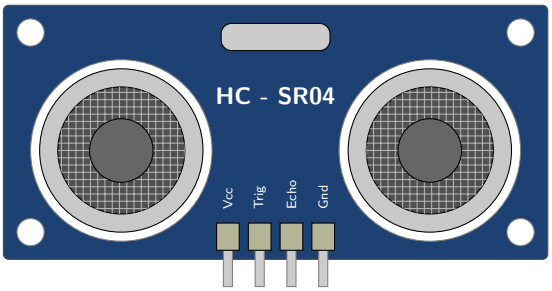


Abbildung 4.1.: Schematische Darstellung des Ultraschallsensors HC-SR04

- Messwinkel: Bis zu 45° bei kurzen Distanzen (< 1 m), Sendekegel von 15° bei maximaler Distanz
- Abmessungen (L x B x T): 45 x 21 x 18 mm

Schaltplan Das Modul besitzt vier Pins 4.1. Der VCC-Pin ist mit der 5V-Spannungsversorgung des Mikrocontrollers verbunden, der GND-Pin (Pin 4) mit dem GND-Pin des Mikrocontrollers. Der TRIG-Eingang (Pin 2) und der ECHO-Ausgang (Pin 3) arbeiten mit TTL-Pegel. Hinweis bei 3.3V-Mikrocontrollern: Da der ECHO-Pin ein 5V-Signal liefert, muss dieser zwingend durch einen Spannungsteiler an den 3.3V-Eingang des Mikrocontrollers angepasst werden [KT-12].

4.4. Bibliothek

4.4.1. Beschreibung

Für die Ansteuerung des Sensors wird die externe C++ Bibliothek **NewPing** von Tim Eckel verwendet [Eck23].

4.4.2. Installation

Die Bibliothek kann direkt über den internen Bibliotheksverwalter der Arduino IDE installiert werden [Eck23].

- **Data types:** Messzeiten in Mikrosekunden und Distanzen in Zentimetern werden als `unsigned int` deklariert, um Speicherplatz zu sparen und negative Distanzen auszuschlieSSen.
- **Data structure:** Der Sensor wird als Objekt der Klasse `NewPing` instanziiert.
- **Data quantity:** Das Softwaredesign ist ressourcenschonend konzipiert. Es können bis zu 15 Ultraschallsensoren gleichzeitig und asynchron über ein einziges Board betrieben werden, bei einer Abtastrate von bis zu 30 Pings pro Sekunde je Sensor.
- **Data quality:** Die Bibliothek bietet integrierte digitale Filter. Die Funktion `pingmedian` verarbeitet mehrere Messwerte zu einem Median, was Rauschen und fehlerhafte Echos effektiv aus dem Datensatz entfernt.

4.4.3. Funktionen

Die wichtigsten Befehle der NewPing-Bibliothek umfassen [Eck23]:

- `NewPing sonar`(TRIGGER_PIN, ECHO_PIN, MAX_DISTANCE 4.1): Konstruktor zur Erstellung des Sensor-Objekts.

- **TRIGGER_PIN (Der Auslöser):** Dieser Anschluss entspricht Pin 2 auf dem HC-SR04 Modul und arbeitet mit einem regulären TTL-Pegel [KT-12]. Um einen neuen Messzyklus auszulösen, muss an diesem Triggereingang ein Signal mit einer fallenden Flanke für mindestens 10 µs anliegen [KT-12]. Als direkte Reaktion auf dieses Signal sendet das Modul selbstständig ein 40 kHz Burst-Signal aus [KT-12]. In dem vorliegenden Code-Beispiel wird dieser Anschluss dem digitalen Pin 12 des Arduino zugewiesen.
- **ECHO_PIN (Der Empfänger):** Dieser Ausgang liegt auf Pin 3 des Sensormoduls und gibt das eigentliche Messergebnis ebenfalls als TTL-Pegel aus [KT-12]. Unmittelbar nach dem Aussenden des Ultraschall-Bursts springt dieser Pin auf einen HIGH-Pegel und wartet auf den Empfang des zurückkehrenden Echos [KT-12]. Sobald das reflektierte Signal detektiert wird, fällt der Ausgang sofort auf einen LOW-Pegel ab [KT-12]. Wird gar kein Echo erkannt (weil der Raum leer ist), verweilt der Pin für exakt 200 ms auf dem HIGH-Pegel, um dem Mikrocontroller die erfolglose Messung zu signalisieren [KT-12]. In der Software ist dieser Pin dem digitalen Pin 11 zugewiesen.
- **MAX_DISTANCE (Die maximale Reichweite):** Rein physikalisch besitzt der HC-SR04 eine messbare Distanz von 2 cm bis zu ca. 300 cm [KT-12]. Soll die maximale Distanz von 3 Metern erreicht werden, muss der Sensor sehr exakt auf das Objekt zielen, und es dürfen keine störenden Gegenstände im Sendekegel von 15° vorhanden sein [KT-12]. Im Programmcode ist `MAX_DISTANCE` eine festgelegte Konstante (hier auf 200 cm gesetzt), die dem `NewPing`-Objekt mitteilt, bis zu welcher Entfernung der Sensor überhaupt auf Objekte reagieren soll [Eck23].

- `sonar.ping()`: Sendet einen Ping und gibt die gemessene Laufzeit in Mikrosekunden (µs) zurück.
- `sonar.ping_cm()`: Führt eine Messung durch und gibt die in Zentimeter (cm) umgerechnete Distanz zurück.
- `sonar.ping_median(int iterations)`: Führt die angegebene Anzahl an Pings durch, verwirft Ausreißer und gibt die mediane Laufzeit zurück.

4.5. Beispiel

In diesem Abschnitt wird ein einfaches, eigenständiges Codebeispiel beschrieben, welches die Funktionsweise und Integration der Bibliothek in C++ verdeutlicht.

4.5.1. Handbuch

1. Verbinden Sie VCC und GND des HC-SR04 mit 5V und GND des Boards [KT-12].
2. Verbinden Sie den TRIG-Pin mit Pin 12 und den ECHO-Pin mit Pin 11.
3. Öffnen Sie die Arduino IDE, kompilieren Sie den Code und laden Sie ihn hoch.
4. Öffnen Sie den Seriellen Monitor (Baudrate 115200). Hier werden nun die Messergebnisse ausgegeben[Ele20].

4.5.2. Funktion vom Beispiel

Das Skript bindet die `NewPing.h` ein. Über `#define` werden Pins und der maximale Messbereich (200 cm) definiert. In der Hauptschleife (`loop()`) stellt `delay(50)` sicher, dass zwischen zwei Pings gewartet wird, um Echosüberlappungen zu verhindern [Eck23]. Anschließend wird `ping_median(5)` aufgerufen. Ist ein Objekt vorhanden, muss die gemessene Zeit in eine Längeneinheit umgewandelt werden. Hierfür kommt der bibliotheksinterne Befehl `US_ROUNDTRIP_CM` zum Einsatz. Dies ist eine vordefinierter Teilerfaktor, mit dem Wert 57. Er basiert auf der physikalischen Schallgeschwindigkeit in der Luft. Da der Schall für einen Zentimeter Wegstrecke etwa 29 Mikrosekunden benötigt und das Signal den Weg doppelt zurücklegen muss (Hin- und Rückweg), entspricht 1 cm Objektabstand exakt einer Signallaufzeit von ca. 57 bis 58 Mikrosekunden. Durch die Division `uS / US_ROUNDTRIP_CM` wird die gemessene Zeit somit effizient und direkt in den korrekten Zentimeter-Abstand umgerechnet [Eck23].

4.5.3. Code

Listing 4.1.: Beispiel Code

```
\begin{verbatim}
// File: Beispielcode.ino
// Standalone script for filtered distance measurement with HC-SR04.
// Uses NewPing library to get a median value from multiple measurements
    to reduce noise.
// Author: Wilko Hinrichs
// Date: 2026-04-27

#include <NewPing.h>

// Digital pin for trigger signal
#define TRIGGER_PIN  12

// Digital pin for echo signal
#define ECHO_PIN     11

// Maximum distance for the sensor (in cm)
#define MAX_DISTANCE 200

// Initialize NewPing object
NewPing sonar(TRIGGER_PIN, ECHO_PIN, MAX_DISTANCE);

void setup() {
  Serial.begin(115200);
  Serial.println("System gestartet. Beginne Messung...");
}

void loop() {
  // Wait between pings (min. 29ms recommended)
  delay(50);

  // Get median of 5 measurements (in microseconds)
  unsigned int uS = sonar.ping_median(5);

  Serial.print("Gefilterte Distanz: ");

  if (uS == 0) {
    Serial.println("AuSSerhalb der Reichweite");
  } else {
    // Convert microseconds to centimeters
    Serial.print(uS / US_ROUNDTRIP_CM);
    Serial.println(" cm");
  }
}
\end{verbatim}
```

../../Code/Ultraschallsensor/Beispielcode/Beispielcode.ino

4.6. Kalibrierung

Die Ultraschallmessung beruht auf der Ausbreitungsgeschwindigkeit von Schall in der Luft. Um das System zu überprüfen wird ein Objekt in einem exakt gemessenem Abstand vor dem Sensor aufgestellt und mit der Ausgabe im seriellen Monitor verglichen. Die Standardberechnung der Bibliothek geht von einer konstanten Schallgeschwindigkeit von $c \approx 343\,\mathrm{m/s}$ bei $20\,°\mathrm{C}$ aus [Fra16] . Da die Schallgeschwindigkeit temperaturabhängig ist, ändert sie sich um ca. $0,6\,\mathrm{m/s}$ pro Grad Celsius [Fra16]. Für hochpräzise Anwendungen muss das System kalibriert werden (z.B. über einen DHT22-Sensor). Die dynamische Berechnung lautet:

$$\text{Distanz} = \frac{\text{Laufzeit}}{2} \times (331,3 + 0,606 \times \text{Temperatur in } °\text{C})$$

.

4.7. Einfacher Code ohne newPing Bibliothek

Der folgende Code demonstriert eine Abstandsmessung ohne Einbindung der `NewPing`-Bibliothek und nutzt `pulseIn()`.

Listing 4.2.: Beispiel Code ohne Bibliothekseinbindung

```
\begin{verbatim}
// File: Beispielcode2.ino
// Standalone script for filtered distance measurement with HC-SR04.
// Uses NewPing library to get a median value from multiple measurements
    to reduce noise.
// Author: Wilko Hinrichs
// Date: 2026-04-27

const int TRIG_PIN = 12; // Pin for trigger signal
const int ECHO_PIN = 11; // Pin for echo signal

void setup() {
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  Serial.begin(115200);
}

void loop() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);

  // 10 microseconds HIGH pulse (ends with falling edge)
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  // pulseIn measures the length of the ECHO pulse
  long duration = pulseIn(ECHO_PIN, HIGH);

  // Calculate distance (travel time in us / 58)
  float distance = duration / 58.0;

  Serial.print("Gemessene Distanz: ");
  Serial.print(distance);
  Serial.println(" cm");
```

```
  // Keep interval (> 20ms)
  delay(50);
}
\end{verbatim}
```

../../Code/Ultraschallsensor/Beispielcode2/Beispielcode2.ino

4.8. Einfache Anwendung

Eine anschauliche Anwendung für den HC-SR04 ist eine automatisierte Einparkhilfe. Fährt ein Fahrzeug heran, liest der Sensor die Distanz. Unterschreitet der `distance`-Wert 50 cm, schaltet das System eine Warn-LED ein. Fällt der Wert auf unter 10 cm, wird ein Buzzer getriggert [Eck23].

Listing 4.3.: Einparkhilfe Beispielcode

```
\begin{verbatim}
// File: EinparkhilfeCode.ino
// Application example: Automated parking assistant.
// System shows distance via LED and warns acoustically on critical
    approach.
// Author: Wilko Hinrichs
// Date: 2026-04-28

#include <NewPing.h>

// Pin configuration
#define TRIGGER_PIN   12
#define ECHO_PIN      11
#define MAX_DISTANCE 200   // Maximum scan range in cm
#define LED_PIN       3    // Yellow warning LED
#define BUZZER_PIN    4    // Acoustic buzzer

// Initialize sensor
NewPing sonar(TRIGGER_PIN, ECHO_PIN, MAX_DISTANCE);

void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);
}

void loop() {
  // Keep measurement interval
  delay(50);

  // Measure distance in cm
  unsigned int distance = sonar.ping_cm();

  Serial.print("Abstand: ");
  Serial.print(distance);
  Serial.println(" cm");

  // Logic check based on distance thresholds
  if (distance > 0 && distance < 10) {
    // Critical area: LED and buzzer active
    digitalWrite(LED_PIN, HIGH);
    digitalWrite(BUZZER_PIN, HIGH);
  }
  else if (distance >= 10 && distance < 50) {
    // Warning area: Only LED active
    digitalWrite(LED_PIN, HIGH);
    digitalWrite(BUZZER_PIN, LOW);
```

```
  }
  else {
    // Safe area: All off
    digitalWrite(LED_PIN, LOW);
    digitalWrite(BUZZER_PIN, LOW);
  }
}
\end{verbatim}
```

../../Code/Ultraschallsensor/EinparkhilfeCode/EinparkhilfeCode.ino

4.9. Tests

4.9.1. Einfacher Funktionstest

Für die Überprüfung der grundlegenden Schaltung und Sensorik wird das Standalone-Skript aus Abschnitt 4.1 (Gefilterte Distanzmessung mit der NewPing-Bibliothek) auf den Mikrocontroller geladen.
Ein harter, planer Gegenstand (z. B. ein Buch) wird in exakt 15 cm Entfernung parallel zur Sensorfront platziert. Anschließend wird der Serielle Monitor der Arduino IDE mit einer Baudrate von 115200 geöffnet. Der ausgegebene Wert muss durch den Aufruf der Funktion `sonar.ping_median(5)` sehr stabil bei 15 cm liegen. Die Schwankung darf dabei die gerätespezifische Auflösung des Moduls von 3 mm [KT-12] kaum überschreiten. Das Skript bestätigt somit, dass das Sende- und Empfangsmodul korrekt arbeiten und die serielle Datenübertragung fehlerfrei funktioniert.

4.9.2. Alle Funktionen testen

Ein vollständiger Systemtest kann wie folgt durchgeführt werden. Hierfür können die Ausgaben des Seriellen Monitors aus dem vorherigen Testcode 4.1 oder das Verhalten der LEDs aus dem Parkassistenten-Code 4.3 beobachtet werden:

- **Minimaldistanz:** Das Objekt wird auf unter 2 cm an den Sensor herangeführt. Da der Sensor nach dem Senden des 200 µs langen Bursts eine kurze Umschaltzeit benötigt [KT-12], kann er Echos im absoluten Nahbereich physikalisch nicht mehr auflösen. Der Monitor sollte hier durch die Bibliothek korrekterweise Außerhalb der Reichweite (Rückgabewert 0) ausgeben [Eck23].

- **Maximaldistanz:** Der Sensor wird in einen leeren, großen Raum gerichtet, sodass das nächste Hindernis mehr als 300 cm entfernt ist. Laut Spezifikation verweilt der ECHO-Pin bei einem fehlenden Echo für 200 ms auf einem HIGH-Pegel [KT-12]. Der Test belegt, dass die NewPing-Bibliothek diesen Hardware-Timeout sicher abfängt, ohne dass sich der Mikrocontroller in einer blockierenden Endlosschleife aufhängt [Eck23].

- **Winkeltest (Abstrahlkegel):** Ein Objekt wird in 50 cm Entfernung positioniert und langsam seitlich aus dem Zentrum bewegt. Überschreitet der Winkel 15° zur Mittelachse, wird die Fläche vom Objekt für die Ultraschallwellen zu klein [KT-12]. Die Messwerte brechen ab, was die seitlichen Grenzen des Erfassungsbereichs bestätigt.

- **Dynamischer Reaktionstest:** Mit dem Code des automatisierten Parkassistenten aus Abschnitt 4.3 wird die Reaktionsgeschwindigkeit der Gesamtanlage getestet. Ein Objekt wird zügig auf den Sensor zubewegt. Die gelbe Warn-LED muss exakt beim Unterschreiten der 50-cm-Marke aufleuchten, und der Buzzer muss ohne spürbare Software-Verzögerung bei exakt 10 cm triggern.

4.10. Weiterführende Literatur

SparkFun Electronics / ElecFreaks (2019): HC-SR04 Ultrasonic Sensor Datasheet, Verfügbar unter: `https://www.sparkfun.com/datasheets/Sensors/Ultrasonic/HCSR04.pdf` [abgerufen am 28.04.2026]

- Diese technische Spezifikation bildet die fundamentale Grundlage für die Ansteuerung. Sie enthält die exakten Timing-Diagramme für das Trigger- und Echo-Signal sowie Angaben zum Öffnungswinkel des Schallkegels, die für die Gehäuseplanung wichtig sind.

Eckel, Tim (2023): Arduino NewPing Library Documentation, GitHub/Bitbucket Repository, Verfügbar unter: `https://bitbucket.org/teckel12/arduino-newping/wiki/Home` [abgerufen am 28.04.2026]

- Eine umfassende Dokumentation zur verwendeten Programmbibliothek NewPing Library von Tim Eckel. Es werden die Vorteile gegenüber herkömmlichen Messmethoden erläutert und die Implementierung von Filtern wie der `ping_median()`-Funktion zur Rauschunterdrückung detailliert beschrieben.

Ekbert Hering und Gert Schönfelder, Sensoren in Wissenschaft und Technik Funktionsweise und Einsatzgebiete, Springer Vieweg, 2023, ISBN: 9783658394905 [**Ekbert:2023**]

- Eine umfassende und aktuelle Einführung in die Grundlagen und Anwendungen von Sensoren in Technik, Biologie und Medizin. Das Buch behandelt neben physikalischen Grundlagen auch Themen wie Schnittstellen, Sicherheit und Signalverarbeitung bei Ultraschallwandlern.

5. SG90 Servomotor

Einleitung

Dieses Kapitel beschreibt den mechanischen Aktor des Systems, den Micro-Servomotor TowerPro SG90. Es werden die technischen Spezifikationen, die hardwareseitige Einbindung sowie die softwareseitige Ansteuerung über die Standard-Bibliotheken detailliert erläutert.

5.1. Allgemeines

Ein Servomotor ist ein geschlossenes mechatronisches System (Closed-Loop), das für präzise Winkelpositionierungen eingesetzt wird. Im Gegensatz zu kontinuierlich drehenden Gleichstrommotoren besteht ein Standard-Servo aus einem DC-Motor, einem Untersetzungsgetriebe, einem Potentiometer zur Positionserfassung und einer Steuerelektronik. Der Mikrocontroller sendet ein Pulsweitenmodulations-Signal (`PWM`), welches die Steuerelektronik auswertet und den Motor exakt auf den gewünschten Winkel dreht und dort aktiv hält. Servomotoren findet man vor allem im RC-Modellbau (z.B. Lenkung, Ruder, Klappen), in Robotergelenken sowie in Schwenkmechanismen für Kameras oder Sensoren, bei denen wiederholgenaue und stabile Positionen erforderlich sind

5.2. Servomotor TowerPro SG90

Der TowerPro SG90 ist einer der weltweit am häufigsten verwendeten Micro-Servomotoren für kleine Projekte. Er zeichnet sich durch sein extrem geringes Gewicht, seine kompakte Bauform und sein Nylon-Zahnradgetriebe aus. Der Aktor verfügt über ein dreipoliges Anschlusskabel: Braun für die Masse (`GND`), Rot für die Versorgungsspannung (`VCC`) und Orange für das PWM-Steuersignal (`Signal`). Der TowerPro SG90 hat einen mechanischen Rotationsspielraum von 180 Grad (0° bis 180°). Gesteuert wird er über ein 50-Hz-PWM-Signal (20 ms Periodendauer). Eine Pulsweite von 1 ms entspricht theoretisch 0°, 1,5 ms entsprechen 90° (Mittelstellung) und 2 ms entsprechen 180°.[Raj12]

5.3. Spezifikation

- **Datenblatt referenzieren:** Die physikalischen und elektrischen Kennwerte beziehen sich auf die offiziellen Spezifikationen von TowerPro und das Datenblatt von Components101.
- **Extrahierte Informationen:**
  - Betriebsspannung: 4,8 V (ca. 5 V)
  - Stellmoment (Stall Torque): 1,8 kg/cm (bei 4,8 V)
  - Stellgeschwindigkeit (ohne Last): 0,1 Sekunden / 60 Grad (bei 4,8 V)
  - Gewicht: 9 Gramm
  - Getriebetyp: Kunststoff (Nylon)

  - Steuersignal: PWM (50 Hz / 20 ms Periode)
  - Dimensionen: ca. 22,2 x 11,8 x 31 mm

- **Schaltplan:** Das rote Kabel (`VCC`) des SG90 wird mit dem 5V-Ausgang (`VUSB`) des Systems verbunden, um ausreichend Leistung für den Motor bereitzustellen. Das braune Kabel (`GND`) wird mit der gemeinsamen Masse verbunden. Das orange Kabel (`Signal`) wird an einen digitalen PWM-fähigen Pin (z.B. Pin 9) des Mikrocontrollers angeschlossen.

  Hinweis: Obwohl der Arduino Nano 33 BLE mit 3,3V-Logik arbeitet, reicht die 3,3V-Signalspannung am PWM-Pin völlig aus, um den 5V-Servomotor zuverlässig zu triggern.

5.4. Bibliothek

5.4.1. Beschreibung

Für die Ansteuerung des SG90 wird in der Arduino Umgebung die integrierte `Servo`-Bibliothek genutzt. Dadurch entfällt für den Programmierer die aufwändige Erzeugung des exakten 50-Hz-PWM-Signals, und der Motor kann bequem über die direkte Vorgabe eines Winkelwertes von 0 bis 180 Grad angesteuert werden.

5.4.2. Installation

Die `Servo`-Bibliothek gehört zu den Kern-Bibliotheken der Arduino IDE und muss nicht separat heruntergeladen werden. Sie ist direkt nach der Installation der Entwicklungsumgebung unter `#include <Servo.h>` verfügbar.

- **Data types:** Die Zielwinkel werden als `int` (Integer/Ganzzahlen) übergeben. Für präzisere Steuerungen können Pulsweiten in Mikrosekunden ebenfalls als `int` übergeben werden.

- **Data structure:** Jeder Servomotor wird im Code als Objekt der Klasse `Servo` deklariert (z.B. `Servo meinServo;`).

- **Data quantity:** Auf Standard Arduino-Boards unterstützt die Bibliothek bis zu 12 Servos gleichzeitig; auf speziellen Architekturen (wie der Mbed-OS Architektur des Nano 33 BLE) kann dies variieren, reicht jedoch für mehrere Aktoren vollkommen aus.

- **Data quality:** Die Bibliothek nutzt Hardware-Timer des Mikrocontrollers (statt softwareseitiger Delays), wodurch das PWM-Signal im Hintergrund extrem stabil und jitter-frei generiert wird, ohne den restlichen Programmablauf zu blockieren.

5.4.3. Funktionen

Die wichtigsten Methoden der Bibliothek sind:

- `attach(pin)`: Verbindet das Servo-Objekt mit einem bestimmten digitalen Pin.

- `write(angle)`: Dreht den Servo auf den entsprechenden Winkel (0° bis 180°).

- `writeMicroseconds(uS)`: Feinkontrolle des Servos durch direkte Angabe der Pulsweite in Mikrosekunden.

- `read()`: Gibt den aktuellen, zuletzt geschriebenen Winkel des Servos zurück.

5.5. Beispiel

Dieser Abschnitt zeigt ein elementares Beispiel (SSweep"), bei dem der Servo kontinuierlich von 0 auf 180 Grad und wieder zurück schwenkt.

5.5.1. Handbuch

SchlieSSen Sie das rote Kabel an 5V, das braune an GND und das orange Signalkabel an Pin 9 des Arduinos an. Kompilieren Sie das Programm und laden Sie es auf das Board. Der Arm des Servomotors sollte nun anfangen, langsam und gleichmäSSig hin und her zu wischen.

5.5.2. An einem Beispiel

Das Skript bindet zunächst die Bibliothek ein und erstellt ein Servo-Objekt. In der `setup()`-Routine wird der Motor an Pin 9 gebunden. In der `loop()`-Schleife sorgen zwei `for`-Schleifen für die Bewegung. Die erste Schleife erhöht den Winkel schrittweise von 0 auf 180, gefolgt von einer kurzen Pause (`delay`) von 15 ms pro Grad, um dem Motor Zeit zu geben, die physikalische Position zu erreichen. Die zweite Schleife führt die Rückwärtsbewegung aus.

5.5.3. Code

Listing 5.1.: Beispiel Code für den TowerPro SG90

```
/**
 * @file     Servomotor.ino
 * @author   Simon Müller
 * @date     2026-05-01
 * @version  1.0
 *
 * @brief    Servo-based radar scanning system.
 *
 * This sketch controls a TowerPro SG90 (or similar) servo to perform
 * a continuous back-and-forth sweep from 0 deg to 180 deg and back.
 * All position commands are absolute angles
 * (0180 deg range).
 */

#include <Servo.h>

Servo radarServo;    ///< Servo object used for radar scanning
int pos = 0;         ///< Current servo position in degrees (0180,
    absolute)

/**
 * @brief Initializes the servo on startup.
 *
 * Attaches the servo object to digital pin 9 and prepares it
 * for subsequent position commands. The servo will interpret
 * all following write() calls as absolute target angles in
 * the range 0ř180ř.
 */
void setup() {
    radarServo.attach(9);  // Attach servo to pin 9
}

/**
 * @brief Main loop for continuous servo motion.
 *
 * The servo is moved in two phases using absolute angle commands:
 * - Phase 1: Sweep from 0ř to 180ř in 1ř steps
 * - Phase 2: Sweep from 180ř back to 0ř in 1ř steps
```

```
 *
 * Each call to radarServo::write(pos) sets an absolute target
 * angle; it does NOT move the servo by a relative offset.
 * With a 15 ms delay between steps, one complete cycle
 * takes approximately 5.4 seconds.
 */
void loop() {
    // Phase 1: sweep from 0ř to 180ř (absolute positions)
    for (pos = 0; pos <= 180; pos += 1) {
        radarServo.write(pos);  ///< Set servo to absolute angle 'pos'
        delay(15);              ///< Wait 15 ms to reach the position
    }

    // Phase 2: sweep from 180ř back to 0ř (absolute positions)
    for (pos = 180; pos >= 0; pos -= 1) {
        radarServo.write(pos);  ///< Set servo to absolute angle 'pos'
        delay(15);              ///< Wait 15 ms
    }
}
```

../../Code/Servomotor/Servomotor.ino

5.5.4. Dateien

Für das Ausführen des Codes wird ausschlieSSlich die Standard-Headerdatei `Servo.h` benötigt, welche in der Arduino IDE vorinstalliert ist.

5.6. Kalibrierung

Oftmals weichen günstige SG90-Motoren mechanisch von den theoretischen PWM-Werten ab (1000 ţs bis 2000 ţs). Wird der Motor mit `write(0)` oder `write(180)` an seine physischen Grenzen gezwungen, kann er brummen, vibrieren und überhitzen (Stalling). Zur Kalibrierung bietet die `attach()`-Funktion optionale Parameter für die minimalen und maximalen Mikrosekunden: `radarServo.attach(9, 500, 2400);`. Diese Werte müssen durch iteratives Ausprobieren (mit `writeMicroseconds()`) an den individuellen Motor angepasst werden, bis der volle 180°-Winkel ohne mechanisches Blockieren erreicht ist.

5.7. Einfacher Code (ohne Bibliothek)

Dieser Code erzeugt das PWM-Signal für den Servo manuell ohne die Servo-Bibliothek. In `setup()` wird Pin 9 als Ausgang konfiguriert. Die `loop()`-Funktion generiert ein 50-Hz-Signal: Der Pin ist 1.500 Mikrosekunden HIGH (entspricht 90° Servo-Position) und anschlie*ss*end 18.500 Mikrosekunden LOW, um die 20-ms-Periode zu vervollständigen. Diese Methode zeigt, wie die Servo-Bibliothek intern arbeitet, ist jedoch blockierend für komplexere Anwendungen sollte die Servo-Bibliothek mit Hardware-Timern verwendet werden.

Listing 5.2.: Code ohne Bibilothek für den TowerPro SG90

```
/**
 * @file ServoManual.ino
 * @brief Manual servo control without library
 *
 * @details Demonstrates servo motor control through direct PWM
   generation
 * using digitalWrite() and delayMicroseconds(). The servo is positioned
 * at approximately 90ř (center position). This method shows how the
 * Servo library works at the hardware level.
 *
 * @author Simon Müller
```

```
 * @date 01.05.2026
 * @version 1.0
 *
 * @section hardware Hardware
 * - Arduino Board (e.g. Uno, Nano)
 * - Servo motor connected to Pin 9
 * - Power supply: 5V for servo (VCC and GND)
 *
 * @section notes Notes
 * This implementation uses blocking delays and is for educational
   purposes.
 * For production code, use the Arduino Servo library instead.
 */

int servoPin = 9;  ///< Pin number for servo control signal

/**
 * @brief Initialization at program startup
 *
 * @details Configures the servo pin as OUTPUT to enable
 * sending PWM signals to the servo motor.
 *
 * @return void
 */
void setup() {
    pinMode(servoPin, OUTPUT);  // Set servo pin as output
}

/**
 * @brief Main loop for continuous PWM generation
 *
 * @details Manually generates a 50 Hz PWM signal (20 ms period).
 * The HIGH pulse of 1.5 ms corresponds to a servo position of approx.
   90ř.
 *
 * PWM Timing:
 * - HIGH phase: 1500 ţs (1.5 ms)  90ř position (center)
 * - LOW phase: 18500 ţs (18.5 ms)  fills up to 20 ms total period
 * - Frequency: 50 Hz (1000000 ţs / 20000 ţs = 50 Hz)
 *
 * @note This method blocks program execution. For more complex
 * applications, the Servo library should be used.
 *
 * @return void
 */
void loop() {
    // Move to approx. 90 degrees (1.5ms HIGH pulse)
    digitalWrite(servoPin, HIGH);     ///< Starts the HIGH pulse
    delayMicroseconds(1500);          ///< Pulse width: 1.5 ms  90ř
    position
    digitalWrite(servoPin, LOW);      ///< Ends the HIGH pulse

    // Fill the remaining 20ms period (50 Hz)
    delayMicroseconds(18500);         ///< LOW phase: 18.5 ms until next
    period
}
```

../../Code/Servomotor/ServomotorohneBib.ino

5.8. Einfache Anwendung

Eine klassische Anwendung für den SG90 ist die Basis eines **Ultraschall-Radarsystems** (wie im vorliegenden Projekt). Der Ultraschallsensor HC-SR04 wird auf der Achse des Servos montiert. Der Servo tastet systematisch in 1-Grad-Schritten die Umgebung vor sich ab (0° bis 180°). Bei jedem Schritt wird eine Entfernungsmessung durchgeführt.

Diese Kombination aus Aktor-Winkel und Sensor-Distanz ermöglicht es der Software (z. B. in Processing), eine zweidimensionale RRadarkarte"der Umgebung auf dem Bildschirm zu zeichnen.

5.9. Tests

5.9.1. Einfacher Funktionstest

Mit dem SSweepCode aus Abschnitt 6.3 lässt sich leicht prüfen, ob der Motor korrekt verkabelt ist. Der Motor muss ruckelfrei und flüssig schwenken. Ein stotternder Motor oder ein Neustart des Arduinos während der Bewegung deutet auf eine unzureichende Stromversorgung (VCC-Abfall) hin.

5.9.2. Alle Funktionen testen

Ein vollständiger Systemtest umfasst:
1. **Grenztest:** Anfahren von 0° und 180°. Der Motor darf in diesen Endlagen nicht surren oder ungewöhnlich warm werden.
2. **Geschwindigkeitstest:** Befehle `write(0)` und sofort danach `write(180)` ohne Delay. Messung, ob der Motor die physikalische Herstellerangabe von 0,3 Sekunden für diesen 180°-Weg einhalten kann.
3. **Präzisionstest:** Verwendung von `writeMicroseconds(1500)` zur exakten Zentrierung, gefolgt von der Überprüfung der Winkelgenauigkeit mittels eines Geodreiecks auf dem Rotorkopf.

5.10. Weiterführende Literatur

Literatur

[1] TowerPro Ltd: *SG90 9g Micro Servo Datasheet & Specifications.*

[2] Arduino Documentation: *Servo Library Reference & Tutorials.*

[3] Components101 (2020): *SG90 Servo Motor Pinout, Datasheet & Wiring.*

[4] ARM mbed OS Documentation: *PWM Configuration for Servo Control on nRF52-based boards.*

Teil III.

Domain Knowledge - Tools

6. Python Tool XY

7. Hardware Tool XY

Teil IV.

Developmenmt

8. Flow Chart Development XY

Teil V.

Monitoring

9. Monitoring XY

Teil VI.

Verification - Evaluation - Conclusion

10. Evaluation/Verification XY

11. To DoX Y

12. Conclusion XY

Teil VII.

Anhang

13. Bill of Material

14. SBOM

requirements.txt

15. Package Example

15.1. Introduction

15.2. Description

15.3. Installation

`pip packageExample`

15.4. Example - Manual

15.5. Example

15.6. Example - Code

15.7. Example - Files

`PackageExample.py`

15.8. Further Reading

Literatur

[Aba+16] M. Abadi u. a. "TensorFlow: A System for Large-Scale Machine Learning". In: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. Savannah, GA: USENIX Association, Nov. 2016, S. 265–283. ISBN: 978-1-931971-33-1. URL: https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi.

Teil VIII.

Hints - Examples for LaTeX - Read, Use and Remove

16. Hinweise

Bitte verwenden Sie **`biber.exe`**.

Wenn Sie ein Symbolverzeichnis erstellen möchten, rufen Sie makeindex auf:
`makeindex %.nlo -s nomencl.ist`

16.1. Allgemeine mathematische Beschreibung Bézier-Kurve

Bézier-Kurven werden mit Hilfe von Bernsteinpolynomen gebildet. Wenn mindestens zwei Punkte sowie zwei Tangenten bekannt sind, können Kontrollpunkte ermittelt werden mit denen ein Kontrollpolygon gebildet wird. An diesem Kontrollpolygon orientiert sich der Verlauf der Kurve. Die Anzahl der Kontrollpunkte ist abhängig vom Grad der Bézier-Kurve. Eine Bézier-Kurve vom Grad n hat n+1 Kontrollpunkte. Die Berechnung der Bézier-Kurve erfolgt über den De-Casteljau-Algorithmus.

Definition. Im Intervall [0; 1] ist das **Bernstein-Polynom n-ten Grades** definiert durch:

$$b_{i,n}(\lambda) = \binom{n}{i}(1-\lambda)^{n-i}\lambda^i, \quad \lambda \in [0,1], \quad i = 0, \ldots, n \tag{16.1}$$

Definition. Der Binomialkoeffizient ist definiert durch:

$$\binom{n}{i} = \frac{n!}{i!(n-i)!}, \quad i = 0, \ldots, n \tag{16.2}$$

Hierbei bilden die Bernstein-Polynome $b_{i,n}$ eine Basis des Vektorraumes $\mathbb{P}^n(I)$ der Polynome höchstens vom Grad n über I. Somit lässt sich jedes Polynom höchstens n-ten Grades eindeutig als Linearkombination schreiben. [Far02]

Beispiel. Bestimmung von Bernstein-Polynome für $n = 3$ gilt:

$$\begin{aligned}
b_{3,0}(\lambda) &= \binom{3}{0}(1-\lambda)^{3-0}\lambda^0 &&= (1-\lambda)^3 \\
b_{3,1}(\lambda) &= \binom{3}{1}(1-\lambda)^{3-1}\lambda^1 &&= 3\lambda(1-\lambda)^2 \\
b_{3,2}(\lambda) &= \binom{3}{2}(1-\lambda)^{3-2}\lambda^2 &&= 3\lambda^2(1-\lambda) \\
b_{3,3}(\lambda) &= \binom{3}{3}(1-\lambda)^{3-3}\lambda^3 &&= \lambda^3
\end{aligned}$$

$b_{3,i}(\lambda)$ mit $i = 0, 1, 2, 3$ sind die kubischen Bernstein-Polynome von Grad 3.

Definition. Gegeben seien die Punkte

$$Q_i = \begin{pmatrix} x_i \\ y_i \end{pmatrix} \quad \text{mit} \quad Q_i \in \mathbb{R}^2, \quad i = 0, 1, \ldots, n$$

Eine **Bézier-Kurve** ist dann definiert durch

$$C(\lambda) = \sum_{i=0}^{n} Q_i \cdot b_{i,n}(\lambda) \tag{16.3}$$

Die Punkte $Q_i, i = 0, \ldots, n$ heiSSen **Kontrollpunkte**.

Die Kontrollpunkte einer Bézier-Kurve bilden das sogenannte Kontrollpolygon.

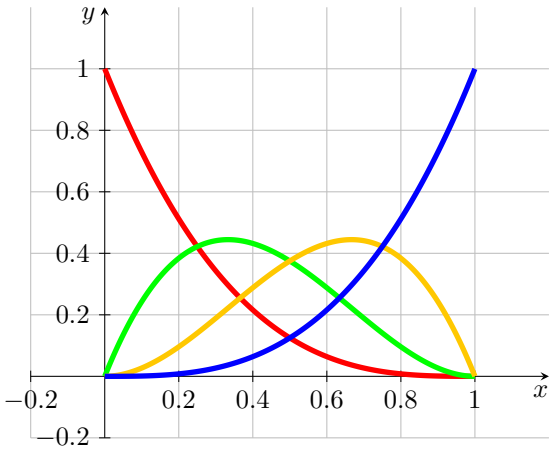


Abbildung 16.1.: Bernstein-Polynom vom Grad 3

Bemerkung. Es sei der Startpunkt P_0 und der Endpunkt P_1, sowie die Tangenten $\vec{t_0}$ und $\vec{t_1}$ gegeben. Die Tangenten sind nicht notwendigerweise normiert.
Die Kontrollpunkte Q_0, Q_1, Q_2 und Q_3 der zugehörigen Bézier-Kurve sind durch folgende Gleichungen gegeben:

$$Q_0 = P_0, \qquad Q_1 = P_0 + \lambda_0 \vec{t_0}, \qquad Q_2 = P_1 - \lambda_1 \vec{t_n}, \qquad Q_3 = P_1 \tag{16.4}$$

[Jak+10]

Beispiel. Gegeben seien

$$P_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad \vec{t_0} = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad \text{und} \quad P_1 = \begin{pmatrix} 6 \\ 1 \end{pmatrix}, \quad \vec{t_1} = \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$

Dann ergibt sich gemäSS Satz 16.4 für Kontrollpunkte der zugehörigen Bézier-Kurve:

$$Q_0 = P_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \qquad Q_1 = P_0 + \vec{t_0} = \begin{pmatrix} 2 \\ 4 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 5 \end{pmatrix},$$

$$Q_2 = P_1 - \vec{t_n} = \begin{pmatrix} 6 \\ 1 \end{pmatrix} - \begin{pmatrix} 1 \\ -2 \end{pmatrix} = \begin{pmatrix} 5 \\ 3 \end{pmatrix}, \qquad Q_3 = P_1 = \begin{pmatrix} 6 \\ 1 \end{pmatrix}$$

Die Bézier-Kurve und ihre Kontrollpunkte sind in der Abbildung 16.2 dargestellt.

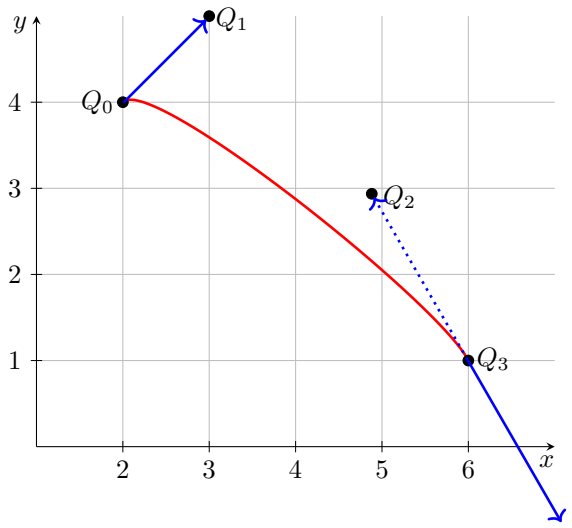


Abbildung 16.2.: Bézier-Kurve zum Beispiel 16.1

17. Verrundung mit einem Kreisbogen

17.1. Gleichungen

Die Verrundung mit einem Kreisbogen ist eine einfache Variante der Glättung von Ecken. Diese Variante ermöglicht die Bearbeitung und die Erzeugung von Dateien gemäSS DIN 66025. Zur Glättung werden stets drei Punkte P_0 und S und P_1 betrachtet, damit ergibt sich ein symmetrisches Hermite-Problem. GemäSS Satz **??** wird vorausgesetzt, dass es sich in der Standardform $HP(L, \alpha)$ befindet.

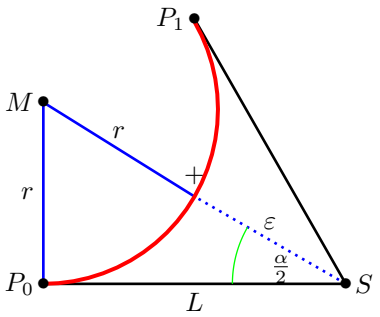


Abbildung 17.1.: Glättung einer Ecke mit Hilfe eines Kreisbogens - Dreieck

Die Abbildung 17.1 stellt die Situation dar. Die Punkte P_0, S und P_1 sind gegeben. Der eingeschlossene Winkel $\alpha = \angle(P_0; S; P_1)$ sowie der Abstand $L = \|\|S - P_0\|\| = \|\|P_1 - S\|\|$ sind in der Grafik dargestellt. Der rote Kreisbogen ist das gewünschte Ergebnis. Der Abstand seines Mittelpunkts M zu S beträgt dann $r + \varepsilon$, wobei r der Radius des Kreisbogens und ε die vorgegebene Toleranz ist. Somit erhalten wir ein rechtwinkliges Dreieck P_0SM, dessen Kantenlängen L, $r + \varepsilon$ und r sind.

GemäSS der Definition des Sinus und des Tangens folgt:

$$\sin\left(\frac{\alpha}{2}\right) = \frac{L}{r+\varepsilon} \qquad \text{und} \qquad \tan\left(\frac{\alpha}{2}\right) = \frac{L}{r}$$

$$\Leftrightarrow \quad \varepsilon = \frac{L}{\sin\left(\frac{\alpha}{2}\right)} - r \qquad \text{und} \qquad r = \cot\left(\frac{\alpha}{2}\right) L$$

Die beiden Gleichung können zusammengefasst werden, so dass sich folgende Bedingungen ergeben:

$$\varepsilon = L \cdot \frac{1 - \cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} \qquad \text{bzw.} \qquad L = \varepsilon \cdot \frac{\sin\left(\frac{\alpha}{2}\right)}{1 - \cos\left(\frac{\alpha}{2}\right)}$$

Damit ergibt sich für den Radius die Gleichung:

$$r = L \cdot \cot\left(\frac{\alpha}{2}\right) = L \cdot \sqrt{\frac{1 - \cos(\alpha)}{1 + \cos(\alpha}} = L \cdot \frac{\sin(\alpha)}{1 + \cos(\alpha)} = L \cdot \frac{P_{1,x}}{P_{1,y}}$$

Der Faktor zum Umrechnung von L und ε kann mit Hilfe trigonometrischer Umformungen vereinfacht werden.

$$\frac{1-\cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} = \frac{1-\sqrt{\frac{1-\cos(\alpha)}{2}}}{\sqrt{\frac{1+\cos(\alpha)}{2}}} = \frac{\sqrt{2}-\sqrt{1-\cos(\alpha)}}{\sqrt{1+\cos(\alpha)}} = \frac{\sqrt{1+\cos(\alpha)}-\sin(\alpha)}{1+\cos(\alpha)}$$

Durch den Vergleich mit dem symmetrischen Hermite-Problem in Standardform ergibt sich dann die folgende Notation:

$$\frac{1-\cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} = \frac{\sqrt{P_{1,x}}-P_{1,y}}{P_{1,x}}$$

Die vorherigen Überlegungen werden nun im nachfolgenden Satz zusammengefasst.

Satz. Es sei ein symmetrisches Hermite-Problem $(P_0, \vec{t}_0, P_1, \vec{t}_1, S, L)$ gegeben. Ohne Einschränkung der Allgemeinheit liegt es in der Standardform

$$\left\{\begin{pmatrix}0\\0\end{pmatrix}, \begin{pmatrix}1\\0\end{pmatrix}, \begin{pmatrix}L+L\cdot\cos(\alpha)\\L\cdot\sin(\alpha)\end{pmatrix}, \begin{pmatrix}\cos(\alpha)\\\sin(\alpha)\end{pmatrix}, \begin{pmatrix}L\\0\end{pmatrix}, L\right\}$$

mit $L \in \mathbb{R}^{>0}$ und $\alpha \in (-\pi; \pi]$ vor.

Dann kann ein Kreisbogen gefunden werden, der die Punkte P_0 und P_1 verbindet, die gleichen Tangentenrichtungen in den beiden Punkten besitzt.

Für die Kreisbogen gilt:

$$r = L\cdot\left|\frac{P_{1,x}}{P_{1,y}}\right|; \quad \phi_0 = -\operatorname{sign}(\alpha)\cdot\frac{\pi}{2}; \quad \phi_1-\phi_0 = \operatorname{sign}(\alpha)\cdot\pi-\alpha = \beta;$$

$$M = P_0 + \operatorname{sign}(\alpha)\cdot r\cdot\vec{t}_0^{\perp} = \operatorname{sign}(\alpha)\cdot r\cdot\begin{pmatrix}0\\1\end{pmatrix}$$

Aus der Vorgabe des Abstand L kann, wie in Satz 17.1 dargestellt, der maximale Fehler berechnet werden. GemäSS der Herleitung ist es auch möglich, den maximalen Fehler ε vorzugeben und daraus den maximalen Abstand L zu bestimmen.

Satz. Es sei ein symmetrisches Hermite-Problem $(P_0, \vec{t}_0, P_1, \vec{t}_1, S, L)$ gegeben. Ohne Einschränkung der Allgemeinheit liegt es in der Standardform

$$\left\{\begin{pmatrix}0\\0\end{pmatrix}, \begin{pmatrix}1\\0\end{pmatrix}, \begin{pmatrix}L+L\cdot\cos(\alpha)\\L\cdot\sin(\alpha)\end{pmatrix}, \begin{pmatrix}\cos(\alpha)\\\sin(\alpha)\end{pmatrix}, \begin{pmatrix}L\\0\end{pmatrix}, L\right\}$$

mit $L \in \mathbb{R}^{>0}$ und $\alpha \in (-\pi; \pi]$ vor.

a) Es sei der maximale Fehler ε vorgegeben. Der maximale Abstand L, für den ein Kreisbogen gemäSS Satz 17.1 existiert, der den Fehler berücksichtigt, ist gegeben durch:

$$L(\varepsilon,\alpha) = \varepsilon\cdot\frac{P_{1,x}}{\sqrt{P_{1,x}}-P_{1,y}} = \varepsilon\cdot\frac{L+L\cdot\cos(\alpha)}{\sqrt{L+L\cdot\cos(\alpha)}-L+L\cdot\cos(\alpha)}$$

b) Bei der Vorgabe des Abstands L ergibt sich der folgende, maximale Fehler:

$$\varepsilon(L,\alpha) = L\cdot\frac{\sqrt{P_{1,x}}-P_{1,y}}{P_{1,x}} = L\cdot\frac{\sqrt{L+L\cdot\cos(\alpha)}-L+L\cdot\cos(\alpha)}{L+L\cdot\cos(\alpha)}$$

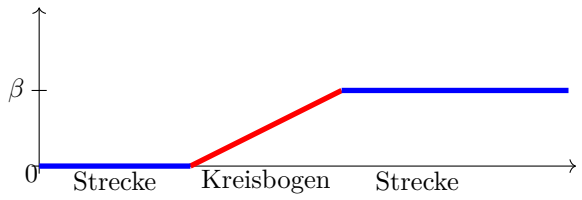


Abbildung 17.2.: Winkeländerung bei einer Verrundung mit einem Kreisbogen

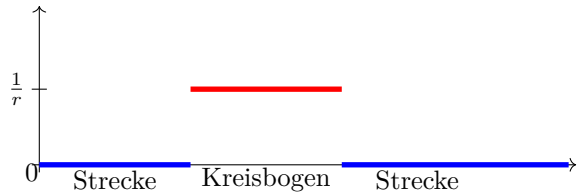


Abbildung 17.3.: Krümmungsverlauf bei der Verrundung mit einem Kreisbogen

Aus diesen Überlegungen ergeben sich Einschränkungen für den Einsatz dieser Strategie, die in der folgenden Bemerkung zusammengefasst sind.

Bemerkung. Folgende Bedingungen zum Anwenden der Strategie eines Kreises müssen erfüllt sein:

a)
$$P_0 \neq P_1$$

b)
$$\vec{t_0} = -\vec{t_1} \quad \Leftrightarrow \quad \alpha = 0$$

c)
$$\vec{t_0} = \vec{t_1} \quad \Leftrightarrow \quad \alpha = \pi$$

17.2. Bewertung

Die Verrundung mittels eines Kreisbogens führt zu einem stetigen Verlauf der Winkeländerung. Die Abbildung 17.2 stellt dies dar.

Durch die Verrundung mit einem Kreisbogen wird ist der Krümmungsverlauf der Kurve nicht stetig. Die Abbildung 17.3 zeigt, dass die Krümmung im Bereich der Strecken 0 ist und auf dem Kreisbogen konstant mit dem Wert $\frac{1}{r}$, wobei r der Radius des eingesetzten Kreisbogens ist. Aufgrund der Formel $a = \frac{v^2}{r}$ für eine Bewegung auf einem Kreisbogen weist der Beschleunigungsverlauf bei einer konstanten Bahngeschwindigkeit Stetigkeitssprüngen an den Übergängen aus.

In Abhängigkeit der Winkeländerung β an der Ecke S und der Toleranz ε kann die maximale Länge der Verkürzung der Strecken berechnet werden. Die Abbildung 17.4 zeigt das zugehörige Diagramm, wobei die Toleranz ε auf den Wert 1 gesetzt ist.

Der Bereich, der zur Verfügung gestellt wird, kann auch vorgegeben werden. In Abhängigkeit des Abstands L vom Eckpunkt S kann die Abweichung ε berechnet

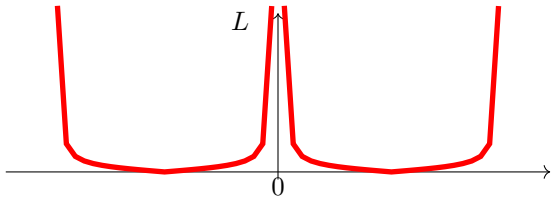


Abbildung 17.4.: Maximaler Bereich für die Verrundung mit einem Kreisbogen - $L(\varepsilon = const, \alpha)$

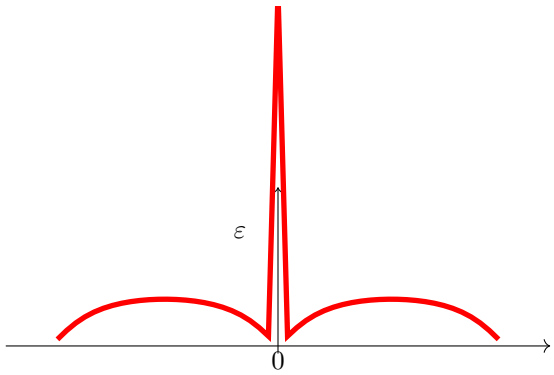


Abbildung 17.5.: Abweichung bei Vorgabe des Abstands L bei der Verrundung mit einem Kreisbogen

werden. Die Abbildung 17.5 stellt die Funktion in Abhängigkeit von L und des Winkels α dar.

Die Abbildungen 17.4 und 17.5 zeigen die Kurven der Länge in Abhängigkeit der Winkeländerung bei einer festen Toleranz und den Fehler in Abhängigkeit der Winkeländerung bei vorgegebenen Länge. Falls die Winkeländerung minimal ist, dann ist der Fehler oder die Länge L extrem. In diesem Fall ist eine Verrundung mittels eines Kreisbogens nicht möglich. Um eine sinnvolle Strategie zu entwickeln, muss eine minimale Winkeländerung vorgegeben werden, ab der eine Verrundung mit einem Kreisbogen erlaubt ist. Ebenso ist eine maximale Winkeländerung zu definieren. Denn bei einer Winkeländerung von $\pm\pi$ handelt sich um eine Kehre. in diesem Fall ist eine Verrundung auch nicht möglich.

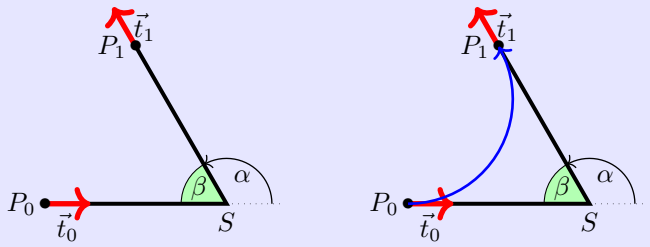
17.3. Beispiele

Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{2}{3}\pi\right) \\ 6.0 \cdot \sin\left(\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{2}{3}\pi\right) \\ \sin\left(\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = \frac{2}{3}\pi$ gegeben.
Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ 3,4641 \end{pmatrix}; \quad r = 3,46412; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{2}{3}\pi$$

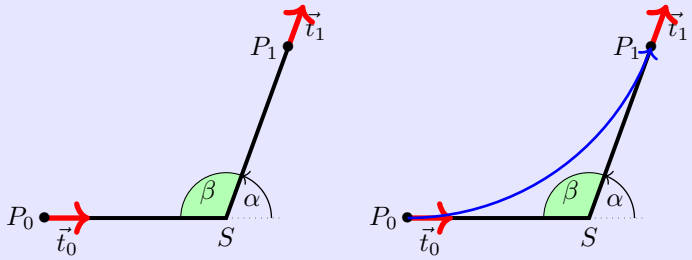


Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{7}{18}\pi\right) \\ 6.0 \cdot \sin\left(\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{7}{18}\pi\right) \\ \sin\left(\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = \frac{7}{18}\pi$ gegeben.
Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ 8,5689 \end{pmatrix}; \quad r = 8,5689; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{7}{18}\pi$$

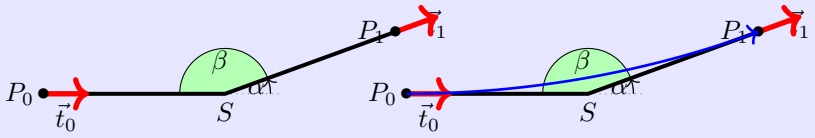


Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{1}{9}\pi\right) \\ 6.0 \cdot \sin\left(\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{1}{9}\pi\right) \\ \sin\left(\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = \frac{1}{9}\pi$ gegeben.
Für den Verrundungskreisbogen folgt:

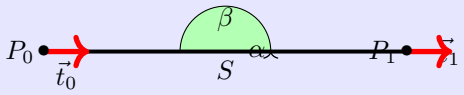
$$M = \begin{pmatrix} 0 \\ 34,0277 \end{pmatrix}; \quad r = 34,0277; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{1}{9}\pi$$



Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 12.0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = 0$ gegeben.
Hier existiert kein Verrundungskreisbogen.

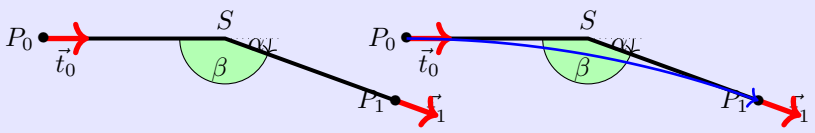


Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(-\frac{1}{9}\pi\right) \\ 6.0 \cdot \sin\left(-\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(-\frac{1}{9}\pi\right) \\ \sin\left(-\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = -\frac{1}{9}\pi$ gegeben.
Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ -34,0277 \end{pmatrix}; \quad r = 34,0277; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = \frac{1}{9}\pi$$

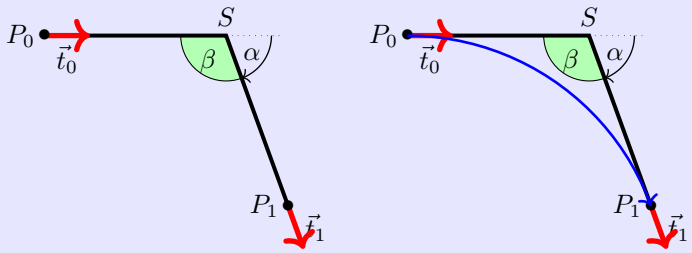


Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(-\frac{7}{18}\pi\right) \\ 6.0 \cdot \sin\left(-\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(-\frac{7}{18}\pi\right) \\ \sin\left(-\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = -\frac{7}{18}\pi$ gegeben.
Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ -8,5689 \end{pmatrix}; \quad r = 8,5689; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = -\frac{7}{18}\pi$$

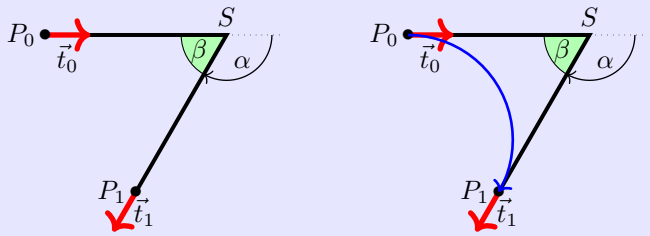


Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(-\frac{2}{3}\pi\right) \\ 6.0 \cdot \sin\left(-\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(-\frac{2}{3}\pi\right) \\ \sin\left(-\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = -\frac{2}{3}\pi$ gegeben.
Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ -3,4641 \end{pmatrix}; \quad r = 3,46412; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = -\frac{2}{3}\pi$$



18. Maple-Dateien

[Wat17c; Wat17d; Wat17b; Wat17e; Wat17a; Wat17f]

18.1. Geometrien mit Maple

Die vorgestellten Algorithmen können gut mit Hilfe von Geometrien dargestellt werden. Dabei können zwei Aspekte untersucht werden. Einerseits ist der Aufwand für eine Umsetzung, die Rechengenauigkeit und die Stabilität eines Algorithmus interessant; andererseits sollen die Verfahren hinsichtlich ihres Nutzens bewertet und verglichen werden. Hierzu wurden Module für das Formelmanipulationssystem Maple [Wat17c] entwickelt.
Maple bietet die Umgebung, Algorithmen einfach und schnell zu implementieren. Darüber hinaus ist die grafische Darstellung mit einfachen Mitteln möglich. Allerdings ist eine einfache Arbeitsmappe von Maple nicht für sehr groSSe Software-Projekte ausgelegt. Hier muss auf die Möglichkeit der Erstellung und Nutzung von Bibliotheken zurückgegriffen werden. Einerseits bietet Maple die Möglichkeit, eigene Bibliotheken, so genannte Module, zu erstellen. Auf diese Weise bleibt man innerhalb der Umgebung und der Syntax von Maple. Dieser Weg wird hier weiter verfolgt. Eine weitere Möglichkeit ist die Verwendung von Bibliotheken, die mittels einer höheren Programmiersprache, z.B. C++, erstellt wurde, den so genannten DLLs.
Im weiteren wird die Erstellung und Verwendung einer Testumgebung mit Hilfe von Module beschrieben. Zunächst werden die Geometrielemente beschrieben, die in verschiedenen Module abgelegt sind, und deren Verwendung. Damit eigene Erweiterungen und Ergänzungen möglich sind, wird dann der Aufbau und die Verwendung eines Moduls in Maple erläutert.

18.2. Verwendete Geometrieelemente

Da es sich um eine Testumgebung für die vorgestellten Algorithmen, werden auch nur Geometrien verwendet, die beschrieben wurden.

- Punkte (`MPoint`)
- Strecken (`MLine`)
- Kreisbögen (`MArc`)
- Bézier-Kurven (`Bezier`)
- Polygonzüge (`MPolygon`)
- Geometrieliste (`MGeoList`)
- Hermite-Probleme (`MHermiteProblem`)
- Symmetrische Hermite-Probleme (`MHermiteProblemSym`)

Für jedes Geometrieelement wurde ein entsprechendes Modul angelegt, dessen Name in der obigen Liste angegeben ist. Die Auflistung, welche Funktionen zur Verfügung stehen, wird in einem weiteren Abschnitt dargestellt.

Bei der Umsetzung eines solchen Projekts wird schnell deutlich, dass bei einer Verwendung von mehreren Geometrieelementen ein klare Datenstruktur und ein wohldefinierter Zugriff auf die Daten unerlässlich ist. Zum Beispiel bei der Darstellung von Geraden ist die Entscheidung zu treffen, ob die Darstellung Punkt-Richtung oder mittels zweier Punkte erfolgen soll. Die Auswahl erfolgt im Allgemeinen in Abhängigkeit des Anwendungsfalls. Hier wird der Weg beschritten, dass aufgrund eines wohldefinierten Zugriffs auf die Daten die Verwendung beider Darstellungen möglich ist.

18.3. Aufbau der Datenstruktur

Die Idee ist, eine möglichst einheitliche und einfache Datenstruktur zu verwenden. Maple bietet zwar die Möglichkeit, Objekte zu definieren. Allerdings ist die Verwendung innerhalb einer prozeduralen Umgebung schwierig. Daher werden alle Daten grundsätzlich als Listen dargestellt. Das erste Listenelement enthält dabei stets eine Kennung des Elements **??**. Dann folgen die Daten, die wiederum Geometrieelemente sein können. Es ist zu beachten, dass der direkte Zugriff auf die Daten nicht unterbunden werden kann; hier ist der Anwender in der Pflicht.

Der Zugriff auf die Daten soll ausschlieSSlich über Prozeduren eines Moduls erfolgen. Im folgenden ist beispielhaft die Datenstruktur für Punkte zu sehen:

```
[MVPOINT,[x,y]]
```

Die Erstellung eines Punkts erfolgt dann über eine Prozedur `New`:

```
NeuerPunkt := MPoint:-New(10,15);
```

Beim Aufruf in der Testdatei wird dann folgendes ausgegeben:

```
TestP0 := [PPoint",[10,15]]
```

Diese Datenstrukturen werden für die Berechnung der Geometrien verwendet. Mit ihnen wird in Funktionen bzw. Prozeduren gearbeitet. Anders als Variablen werden die Datenstrukturen und Prozeduren hier global und nicht lokal definiert. Unter dem Befehl `export` werden die Prozeduren zu Beginn des Moduls deklariert und sind so auch auSSerhalb des Moduls verwendbar. Soll beispielsweise eine Datenstruktur aus `MPoint` in einem anderen Modul oder auSSerhalb der Module verwendet werden wird sie wie folgt aufgerufen:

```
P0 := MPoint:-New(x,y);
```

Zusätzlich zu den Modulen für die Geometrien gibt es ein Modul (MConstant) zum Speichern von Konstanten und Namen. Dort ist für `MVPOINT` z.B. „Point “ gespeichert oder z.B. eine Konstante für den Vergleich auf Null für reelle Zahlen.
Zuletzt gibt es die Testdatei, welche kein Modul ist, in der die Module geladen, aufgerufen und in ihrer Funktion getestet werden. Zu dieser Datei später im Punkt „1.4 Maple Testdatei “ mehr.

18.4. Struktur eines Moduls

In dem folgenden Abschnitt geht es um den Zweck von Modulen und deren Struktur eingegangen.

Es geht in dem Projekt darum die oben genannten Geometrien in einer Datenstruktur zu erfassen und sie in Maple darzustellen. Für die letztendliche Darstellung sind die Berechnungen und Formeln, die verwendet werden müssen, allerdings hinderlich. Module eignen sich sehr gut dafür die Berechnungen, die in Funktionen erfolgen,

zusammenzufassen und zu verstecken. So kann in Maple auf die wichtigen Funktionen zugegriffen werden, ohne dass man sieht, was in diesen steht.

Beispiel:
Die Funktion `Angle` aus dem Modul `MPoint`: Funktion zur Berechnung eines Winkels zwischen Ortsvektor und x-Achse:

```
Angle := proc(P)
   local alpha, x, y;
   x := GetX(P);
   y := GetY(P);
   alpha := 0;
   if abs(x) < 0.00001
      then
         alpha := 3*Pi/2;
      else
         alpha := Pi/2;
      end if;
   else
      alpha := arctan(y/x);
      if x < 0
      then
         alpha := alpha + Pi;
      else
         if y < 0
         then
            alpha := alpha + 2*Pi;
         end if;
      end if;
   end if;
   return factor(alpha);
end proc;
```

Diese Funktion ist lang, stellt aber nur einen kleinen Teil des Programms für die Darstellung dar, um die es geht. Deshalb steht sie in dem Modul und kann so extern mit einem einzigen Befehl aufgerufen werden:

```
Winkel := MPoint:-Angle(P)
```

Im Folgenden Abschnitt wird die Struktur eines Moduls beschrieben. Da Maple hier sehr vielfältige Möglichkeiten bietet, findet eine Beschränkung statt. Es wird nur die Struktur der verwendeten Module, hier anhand des Moduls`MPoint`, beschrieben. Für tiefergehende Möglichkeiten wird hier auf das Handbuch von Maple [Wat17c] verwiesen.

<u>Struktur:</u>

Das Modul muss zunächst gestartet werden. Dies erfolgt mit dem Namen (hier immer ein groSSes M und die Geometrie) des Moduls und dem folgenden Befehl:

```
MPoint := module()
```

Danach müssen, ähnlich wie in Prozeduren, die Variablen und Funktionen definiert werden. Dabei kann man entweder lokal oder global deklarieren. Werden die Variablen

oder Prozeduren nur in dem Modul gebraucht und verändert, erfolgt die Deklaration mit dem Befehl `local` wie folgt:

```
local Variablennamen, mit, Komma, getrennt;
```

Sollen die Variablen oder Prozeduren auch auSSerhalb des Moduls verwendbar sein, wird mit `export` dies definiert:

```
export Funktionsnamen, mit, Komma, getrennt;
```

Daraufhin folgt die Angabe der Optionen:
`option package;`,
die Beschreibung des Moduls:
`description “Selbstgewählte Modulbeschreibung z.B. Modul für Punkte “;`
und die Initialisierung des Moduls:

```
ModuleLoad := proc
  MVPOINT:=MConstant:-GetPoint();
  print("Modul MPoint ist geladen");
end proc;
```

Ab hier wird programmiert wie sonst in Maple auch. Man verwendet die vorher definierten Prozedur- und Variablennamen und programmiert mit Befehlen die man auSSerhalb eines Moduls in Maple auch verwendet. Wichtig zu wissen ist, dass bei Modulen jede Funktion ständig abgerufen werden kann, die Reihenfolge der Funktionen also keine Rolle spielt.

Um zuletzt das Modul zu beenden, nutzt man den folgenden Befehl:

```
end module;
```

18.4.1. Genereller Aufbau eines Moduls

Zusammenfassend haben die Module also folgendes Gerüst:

```
Modulname := module()

   local Namen, mit, Komma, getrennt;

   export Namen, mit, Komma, getrennt;

   global Namen, mit, Komma, getrennt;[1]

   option package;

   description "Selbstgewählte Modulbeschreibung";


   ModuleLoad := proc()[2]
      MVPOINT:=MConstant:-GetPoint();
      print("Modul MPoint ist geladen");
   end proc;

 Procedure1 := proc (Übergabeparameter)
```

[1] möglich, aber in den Modulen nicht verwendet
[2] Beispiel `MPoint`

```
      inhalt;
   end proc;

   Procedure2 := proc (Übergabeparameter)
      inhalt;
   end proc;


   ...

end module;
```

18.4.2. Sicherung eines Moduls

Die Verwaltung von Module wird von Maple automatisch durchgeführt. Da aber die Module weitergegeben werden und einzeln zu bearbeiten sind, müssen einige Einstellungen vorgenommen werden. Daher wird bei jeder Maple-Datei des Projekts folgender Präfix verwendet:

```
1 restart;
2 with(LibraryTools);
3 lib := C:/FH/Tools/Maple/MyLibs/Blending.mla";
4 march('open', lib);
5 ThisModule := 'MArc';
```

Die 1.Zeile initialisiert das System. Die nächsten 3 Zeile ermöglichen das Arbeiten mit Archiven. In der zweiten Zeile werden die Werkzeuge geladen, so dass die Variable `lib` belegt werden kann. Alle Module werden in einem Archiv abgelegt; in diesem Beispiel ist es die Datei `Blending.mla` im Verzeichnis `C:/FH/Tools/Maple/MyLibs/`. Das Kommando `ThisModule := 'MArc';` enthält den Namen des aktuellen Moduls.

Ein Modul wird dann mittels des Befehls `savelib('ModuleName')` gespeichert. Es wird dann eine Datei `ModuleName.mla` angelegt. Je nach Konfiguration von Maple ist der eingestellte Pfad, wo die Datei automatisch angelegt wird, nicht beschreibbar. Dann kann man das System so konfigurieren, dass die mla-Datei im aktuellen Verzeichnis oder in einem Verzeichnis der eigenen Wahl abgelegt wird.

Falls der obige Vorspann für eine Maple-Datei verwendet wird, genügt zum Speichern des Moduls

```
savelib(ThisModule, lib);
```

18.4.3. Verwendung eines Moduls

Ein Modul, das abgespeichert wurde, kann nun in anderen Maple-Worksheets verwendet werden. Dazu wird der Befehl

```
with(ModuleName)
```

verwendet. Falls man einen speziellen Pfad verwendet hat, so kann Maple die Datei `ModuleName.mla` nicht finden. Dann muss der Pfad bekannt gemacht werden, z.B.:

```
savelibname := c:/Maple/MyLibs";", savelibname;
```

Nun kann man auf die Prozeduren des Moduls zugegriffen werden. Eine Übersicht über alle exportierten Prozeduren wird durch den Befehl

```
Describe(ModuleName)
```

angezeigt. Dabei wird eine List der Namen inklusiver der Namen der Übergabeparameter dargestellt. Falls eine Prozedur mit der Feld `description` ausgestattet ist, so wird dieser Text zusätzlich wiedergegeben.

18.4.4. Erstellung eines Maple-Moduls

Die Erstellung eines eigenen Moduls ist schnellt erledigt, falls man den obigen Rahmen verwendet. Allerdings sollte man vorher einige Überlegungen anstellen und einen Standard pflegen.

Bevor ein eigenes Modul erstellt wird, sollte das Datenmodell und die zugehörigen Prozeduren erarbeitet sein. Ein Programmablaufplan leistet hier gute Dienste. Die zentrale Aufgabe des Moduls ist in der Regel schnell ermittelt. Zusätzlich sollten aber folgende Regeln eingehalten werden.

Regel 1 Grundsätzlich erhält sowohl das Modul als auch jede Prozedur eine Beschreibung. Diese beschränkt sich nicht auf das optionale Argument `description`,

sonder wird jeder Prozedur vorangestellt. Die Beschreibung enthält grundsätzlich die Beschreibung der Aufgabe. Dabei werden auch die Voraussetzung genannt bzw. eine Fehlerbhandlung beschrieben. Dann folgt die Beschreibung aller Eingabeparameter, deren Funktion und deren Datenstruktur. Der Rückgabewert wird anschlieSSend beschrieben.

Regel 2 Jedes Modul erhält eine Prozedur `Version()`, die die aktuelle Versionsnummer zurückliefert.

Regel 3 Daten sind nicht global. Um auf Daten zugreifen zu können, werden Proceduren `Set*` und `Get*` zur Verfügung gestellt. Auch innerhalb der Prozeduren eines Moduls werden diese Prozeduren verwendet.

Regel 4 Für jede Prozedur wird mindestens eine Testfunktion geschrieben. Anhand der Testfunktion kann die Verwendung und ggfs. Besonderheiten dargestellt werden.

18.5. Funktionen der Module

Im folgendem werden die einzelnen Module aufgeführt. Die Datenstruktur jedes Moduls und alle Funktionen mit zugehöriger Aufgabe werden genannt.

18.5.1. Modul `MPoint`

`MPoint` ist ein Modul für Punkte und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur `New` für einen Punkt ist eine Liste, die wie folgt aufgebaut ist:

```
[MVPOINT, [x,y]]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name PPointünd die Elemente sind die x- und die y-Koordinate für einen Punkt. Es wird hier kein Unterschied zwischen Punkt und Vektor gemacht. Ein Punkt kann auch als Vektor vom Koordinatenursprung zum Punkt gesehen werden.

Über die Get-Funktionen `GetX` und `GetY` kann man die Koordinaten des Punktes erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Punkten/Vektoren dann gerechnet werden.

MPoint		
E	New	Datenstruktur für einen Punkt
E	GetX	Lesen der x-Koordinate
E	GetY	Lesen der y-Koordinate
E	Angle	Berechnung des Winkels zwischen x-Achse und dem Punkt
E	Add	Errechnet eine Linearkombination zweier Punkte/Vektoren
E	Sub	Errechnet die Differenz zweier Punkte/Vektoren
E	Cos	Errechnet den Cosinus zwischen zwei Vektoren
E	Sin	Errechnet den Sinus zwischen zwei Vektoren
E	Scale	Skaliert einen Vektor mit einem Faktor
E	Perp	Errechnet einen Vektor, der orthogonal zum angegebenen Vektor ist
L	IllustrateXY	Plot Funktion zur Darstellung eines blauen Punktes
E	Illustrate	Darstellung eines blauen Punktes
E	Plot	Darstellung eines grünen Punktes
E	Plot2D	Darstellung eines Punktes mit eigenen Optionen
E	Length	Errechnet den Abstand des Punktes zum Koordinatenursprung
E	Uniform	Normiert den Vektor auf die Länge 1
E	LinetoVector	Errechnet Vektor aus einer Strecke
E	Distance	Errechnet den Abstand zwischen zwei Punkten

18.5.2. Modul MLine

MLine ist ein Modul für Strecken und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur New für eine Strecke ist eine Liste, die wie folgt aufgebaut ist:

```
[MVLINE, [P0,P1]]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „Line“ und die Elemente sind der Start- und der Endpunkt für eine Strecke. Die Datenstruktur NewPointVerctor ist ebenfalls eine Datenstruktur für eine Strecke, allerdings wird die Strecke hier über einen Startpunkt und einen Richtungsvektor definiert.
Über die Get-Funktionen StartPoint und EndPoint kann man Start- und Endpunkt der Strecke erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Strecken dann gerechnet werden.

MLine		
E	New	Datenstruktur für eine Strecke (Zwei-Punkt-Form)
E	NewPointVector	Schaffung einer Strecke (Punkt-Richtungs-Form)
E	StartPoint	Lesen des Startpunktes
E	EndPoint	Lesen des Endpunktes
E	Position	Errechnen eines Punktes der auf der Strecke liegt
E	Plot2D	Darstellung eines Teils der Strecke ausgehend vom Startpunkt
E	Plot2DTangent	Darstellung eines Punktes mit Tangente
E	Plot2DTangentArrow	Darstellung eines Punktes mit Tangente (als Pfeil)
E	LineLine	Errechnung des Schnittpunktes zweier Geraden
E	AngleLineLine	Errechnung des Winkels zwischen zwei Geraden

18.5.3. Modul MArc

MArc ist ein Modul für Kreisbögen und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur New für einen Kreisbogen ist eine Liste, die wie folgt aufgebaut ist:

```
[MVARC, [mx,my,r,phi,alpha]]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name Ärcünd die Elemente sind die x- und y-Koordinate für den Mittelpunkt, der Radius, Der Startwinkel und die Winkeländerung für einen Kreisbogen.
Über die Get-Funktionen GetM, GetMX, GetMY, GetR, GetPhi, und GetAlpha kann man die Daten für einen Kreisbogen erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Kreisbögen dann gerechnet werden.

MArc

E	**New**	Datenstruktur für einen Kreisbogen
E	**GetMX**	Lesen der x-Koordinate des Mittelpunktes
E	**GetMY**	Lesen der y-Koordinate des Mittelpunktes
E	**GetR**	Lesen des Radius
E	**GetPhi**	Lesen des Startwinkels
E	**GetAlpha**	Lesen der Winkeländerung
E	**GetM**	Lesen des Mittelpunktes
E	**Position**	Errechnung eines Punktes auf dem Kreisbogen
E	**Plot2D**	Darstellung eines Teils des Kreisbogens ausgehend vom Startwinkel
E	**Blend**	Errechnung eines Kreisbogens aus einem symmetrischen Hermite-Problem

18.5.4. Modul MBezier

Die Datenstruktur New für ein Polygon ist eine Liste, die wie folgt aufgebaut ist:

```
[MVBEZIER, [PointList]]
```

Innerhalb des Modules MBezier existieren die in folgender Tabelle aufgeführten Prozeduren.

MBezier

E	**New**	Manuelle Eingabe von Kontrollpunkten für eine Bézier-Kurve
E	**Version**	Ausgabe der Verions
E	**BlendCurvature**	Bestimmung von Kontrollpunkten aus symmetrischen Hermite-Problem
E	**BlendCurvatureEpsilon**	Bestimmung von Kontrollpunkten aus symmetrischen Hermite-Problem mit vorgegebenen Fehler
E	**Position**	Position auf der Bézier-Kurve
E	**GetTheta**	Lesen des Winkels
E	**GetEpsilon**	Lesen des Toleranz
E	**GetControlPoint**	Lesen der Kontrollpunkte
E	**Plot2D**	Darstellung der Bézier-Kurve
E	**PlotControlPoints**	Darstellung aller Kontrollpunkte

18.5.5. Modul MPolygon

MPolygon ist ein Modul für Polygonzüge und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur New für ein Polygon ist eine Liste, die wie folgt aufgebaut ist:

```
[MVPOLYGON, [PointList]]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name PPolygonünd die Elemente sind beliebig viele Punkte.

Über die Get-Funktionen GetPoint und GetN kann man die Anzahl der Punkte und die Punkte selbst erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Punkten bzw. dem Polygonzug dann gerechnet werden.

MPolygon		
E	**New**	Datenstruktur für eine Punkteliste
E	**GetPoint**	Lesen des i-ten Punktes aus der Punkteliste
E	**GetN**	Bestimmung der Anzahl der Punkte in der Punkteliste
E	**Length**	Bestimmung der euklidischen Länge des Polygonzuges
E	**Position**	Berechnung eines Punktes auf dem Polygonzug
E	**Tangents**	Berechnung der Tangenten
L	**Plot2DAll**	Plotliste aller Punkte
E	**Plot2D**	Darstellung aller Punkte als Polygonzug
E	**Plot2DTangent**	Darstellung des Polygonzuges mit Tangenten
E	**PlotPoints**	Darstellung aller Punkte

18.5.6. Modul MGeoList

MGeoList ist ein Modul für eine Geometrieliste und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur New für eine Geometrieliste ist eine Liste, die wie folgt aufgebaut ist:

```
[MVGEOLIST, []]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „GeoList“ und die Elemente sind beliebig viele einzelne Geometrien.

Über die Get-Funktionen GeoGeo und GetN kann man das i-te Element der Liste und die Anzahl der Elemente in der Liste erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Geometrien bzw. der Liste dann gerechnet werden. In den Funktionen Length, Plot2DAll, Plot2D und Position werden die Funktionen in sich selber aufgerufen. Dies ist möglich da die Funktionen unabhängig voneinander funktionieren.

MGeoList		
E	New	Datenstruktur für eine Geometrieliste
E	Append	Anfügen eines Geometrieelements in die Datenstruktur
E	Prepend	Einfügen eines Geometrieelements als erstes Element der Liste
E	Replace	Ersetzen eines Geometrieelements durch ein Anderes
E	GetN	Bestimmung der Anzahl der Geometrieelemente in der Liste
E	GeoGeo	Lesen des i-ten Geometrieelements
E	Length	Errechnet die euklidische Länge der Geometrieelemente
E	Position	Berechnung eines Punktes auf der Geometrie
L	Plot2DAll	Plot Funktion für die Darstellung der Geometrieelemente
E	Plot2D	Darstellung eines Teils der Geometrieliste ausgehend vom ersten Element

18.5.7. Modul MHermiteProblem

MHermiteProblem ist ein Modul für Hermite-Probleme und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur New für eine Strecke ist eine Liste, die wie folgt aufgebaut ist:

```
[MVHERMITEPROBLEM, [P0,T0n,P1,T1n]]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „HermiteProb“ und die Elemente sind zwei Punkte mit zugehörigen Tangenten (Strecken).
Über die Get-Funktionen StartPoint, EndPoint, StartTangent und EndTangent kann man die Punkte und ihre Tangenten erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Daten für das Hermite-Problem dann gerechnet werden.

MHermiteProblem		
E	New	Datenstruktur für ein Hermite-Problem
E	StartPoint	Lesen des Startpunktes
E	EndPoint	Lesen des Endpunktes
E	StartTangent	Lesen der Starttangente
E	EndTangent	Lesen der Endtangente
E	Plot2D	Darstellung des Hermite Problems

18.5.8. Modul MHermiteProblemSym

MHermiteProblemSym ist ein Modul für symmetrische Hermite-Probleme und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur New für ein symmetrisches Hermite-Problem ist eine Liste, die wie folgt aufgebaut ist:

```
[MVHERMITEPROBLEMSYM, [P0,T0n,P1,T1n,S,L]]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „SymHermiteProb“ und die Elemente sind zwei Punkte mit zugehörigen Tangenten, ihr Schnittpunkt, und der Abstand der Punkte zum Schnittpunkt.

Über die Get-Funktionen StartPoint, EndPoint, StartTangent, EndTangent, CrossPoint und ParameterL kann man die Daten erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Daten für das symmetrische Hermite-Problem dann gerechnet werden.

MHermiteProblemSym		
E	New	Datenstruktur für ein Symmetrisches Hermite-Problem
E	StartPoint	Lesen des Startpunktes
E	EndPoint	Lesen des Endpunktes
E	StartTangent	Lesen der Starttangente
E	EndTangent	Lesen der Endtangente
E	ParameterL	Lesen des Abstandes
E	CrossPoint	Lesen des Schnittpunktes
E	Plot2D	Darstellung des symmetrischen Hermite Problems
E	Create	Erzeugung eines Sym. Hermite-Problems durch 3 Punkte
E	BlendArc	Verrundung des Eckpunktes durch einen Kreisbogen

18.5.9. Modul MConstant

MConstant ist ein Modul zum Speichern von festen Konstanten und Namen zur Kennung der Datenstrukturen. In der lokal deklarierten Funktionen, beginnend mit CV, werden die Konstanten zur Deklaration der verschiedenen Geometrien definiert. Dies sind Namen zur Erkennung der Geometrieelemente in der Testdatei. In den global deklarierten Funktionen, beginnend mit Get, erfolgt die Rückgabe der Namen zur Kennung der Geometrieelemente.

MConstant		
L	NULLEPS	Konstante zum Vergleich auf Null
L	CVPOINT	Konstante für Geometrieelemente: Point
L	CVLINE	Konstante für Geometrieelemente: Line
L	CVARC	Konstante für Geometrieelemente: Arc
L	CVPOLYGON	Konstante für Geometrieelemente: Polygon
L	CVGEOLIST	Konstante für Geometrieelemente: GeoList
L	CVHERMITEPROBLEM	Konstante für Geometrieelemente: HermiteProb
L	CVHERMITEPROBLEMSYMMETRIC	Konst. für Geometrieelemente: SymHermiteProb
L	CVBIARC	Konst. für Geometrieelemente: Biarc
E	GetNullEps	Rückgabe des Nullvergleichs
E	GetPoint	Kennung für Punkte
E	GetLine	Kennung für Strecken
E	GetArc	Kennung für Kreisbögen
E	GetPolygon	Kennung für Polygone
E	GetGeoList	Kennung für Geometrielisten
E	GetHermiteProblemSymmetric	Kennung für symmetrische Hermite-Probleme
E	GetHermiteProblem	Kennung für Hermite-Probleme
E	GetBiarc	Kennung für Biarcs

18.5.10. Modul MGeneralMath

MGeneralMath ist ein Modul für allgemeine mathematische Funktionen und arbeitet mit den Datenstrukturen und Prozeduren.

MGeneralMath		
E	MPoint	Datenstruktur für einen Punkt
E	MPointX	Lesen der x-Koordinate für einen Punkt
E	MPointY	Lesen der y-Koordinate für einen Punkt
L	MPointIllustrateXY	Plotstruktur für einen blauen Punkt
E	MPointIllustrate	Darstellung eines blauen Punktes
E	MPointPlot	Darstellung eines grünen Punktes
E	MLine	Datenstruktur für eine Strecke
E	MLineStartPoint	Lesen des Startpunktes für eine Strecke
E	MLineEndPoint	Lesen des Endpunktes für eine Strecke
E	MPointOnLine	Berechnung eines Punktes auf der Strecke
E	MLinePlot2D	Darstellung des Teils einer Strecke beginnend beim Startpunkt
E	MLineLine	Errechnen des Schnittpunktes zweier Strecken

18.5.11. Modul Biarc

Datenstruktur

Voraussetzungen und Festlegungen:

Der Biarc soll ein dafür verwendet werden ein Hermite-Problem zu lösen. Ein Hermite Problem ist wie folgt definiert:

Gegeben seien zwei Punkte P_0 und P_1 mit zugehörigen normierten Tangenten $\vec{t}_0$ und $\vec{t}_1$. Der Biarc muss die Punkte so verbinden, dass die Tangenten des Start- und des Endpunktes des Biarcs mit den Tangenten des Hermites-Problems übereinstimmen.

Datenstruktur:

Die Datenstruktur `New` für einen Biarc muss zwei Kreisbögen enthalten.

Benötigt wird also die Datenstruktur für einen Kreisbogen: `MArc:-New`

Diese enthält wiederum die Koordinaten für den Mittelpunkt, den Radius, den Startwinkel und die Winkeländerung.

18.5.12. Modul MBiarc

`MBiarc` ist ein Modul für Biarcs und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur `New` für einen Biarc ist eine Liste, die wie folgt aufgebaut ist:

```
[MVBIARC, [Arc0, Arc1]]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „Biarc“ und die Elemente sind zwei Kreisbögen.

Über die Get-Funktionen `GetArc0` und `GetArc1` kann man die beiden Kreisbögen für den Biarc erfassen. Mit den unten aufgeführten Funktionen kann man die Daten für den Biarc errechnen.

MBiarc		
E	New	Datenstruktur für einen Biarc
E	GetArc0	Lesen des ersten Kreisbogens
E	GetArc1	Lesen des zweiten Kreisbogens
E	Circle	Berechnung des Kreises K_J aus den Daten des Hermite-Problems
E	Plot2DCircle	Darstellung des Kreises K_J
E	angle	Errechnet den Winkel vom Kreismittelpunkt zu Punkten auf dem Kreis
E	Plot2D	Darstellung des Biarcs
E	ConnectionPoint	Errechnung des VerbindungsPunktes J (Equal Chord)
E	TangentTj	Errechnen der Tangente an J
E	Tangent Biarc	Drehung von Tj für den Biarc
E	BiarcCenter	Errechnen der Mittelpunkte der Kreisbögen des Biarcs
E	Biarc	Errechnen des Biarcs
E	Position	Ermitteln eines Punktes auf dem Biarc
E	Blend	Errechnen des Biarcs nur aus dem Hermite-Problem

18.6. Programmablaufplan

18.6.1. Gesamauf

18.6.2. Verrundung der Kurve

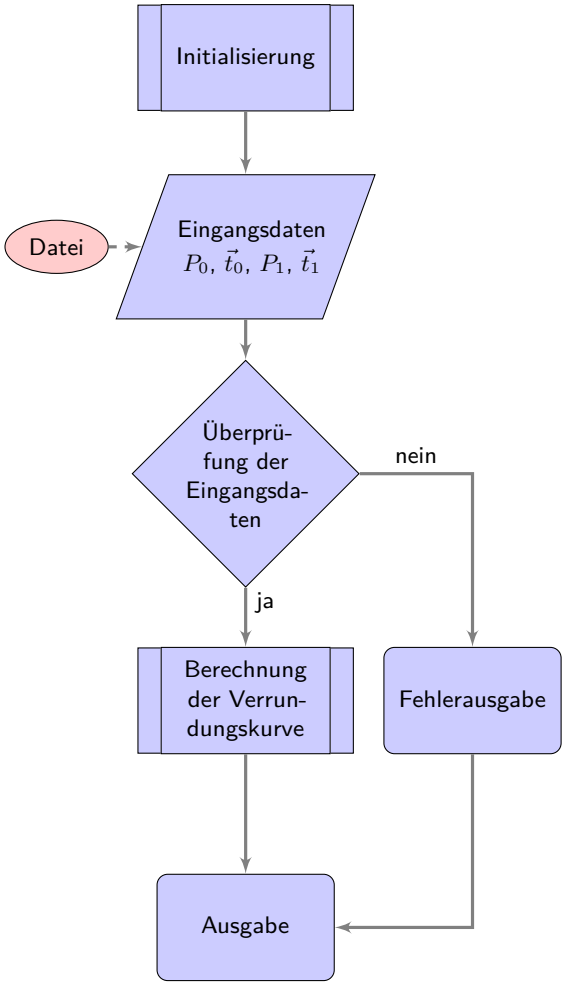


Abbildung 18.1.: Programmablaufplan „Eckenverrrundung“

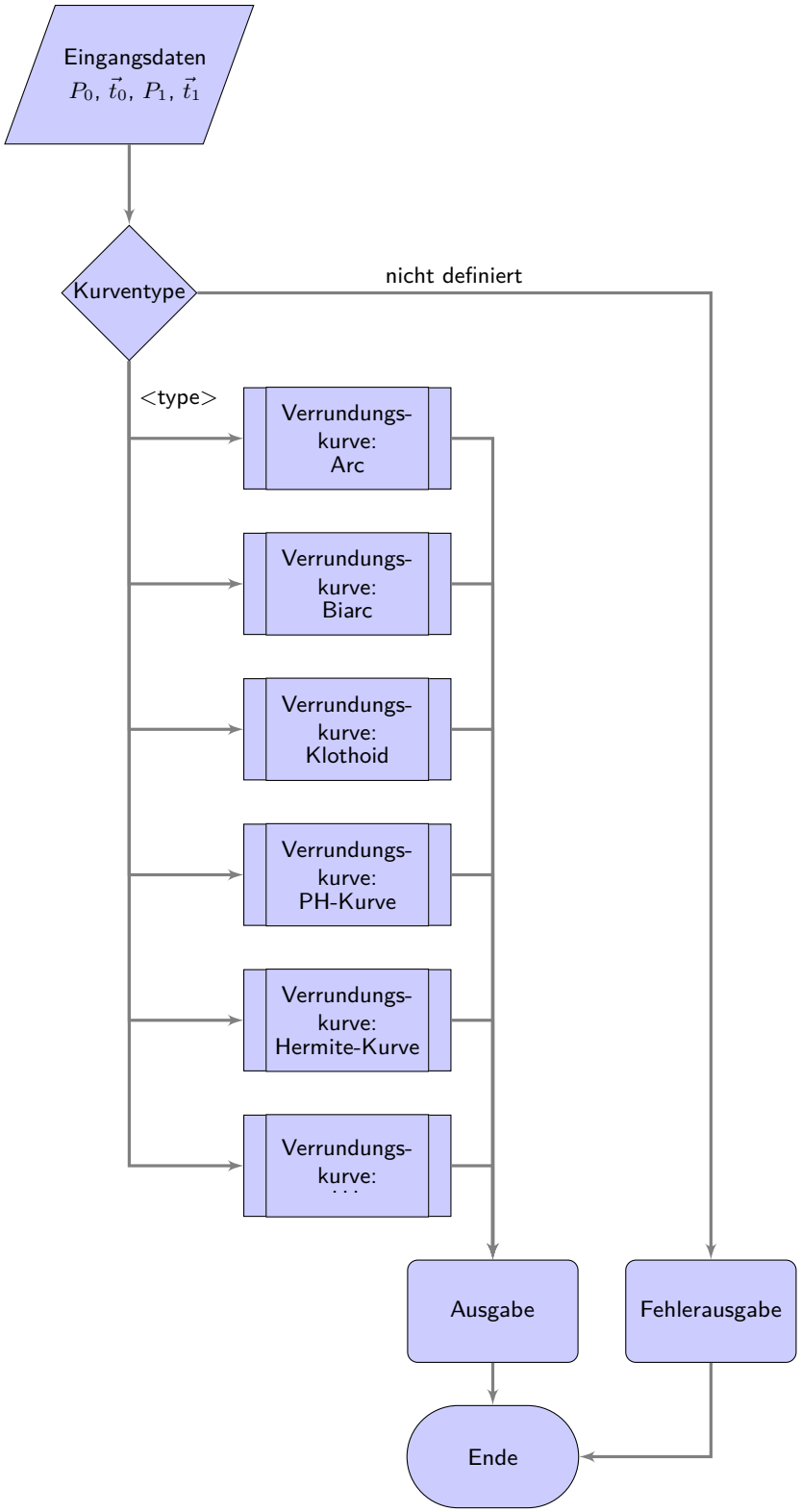


Abbildung 18.2.: Programmablaufplan „Auswal der Verrundungsstrategien“

19. Representation of Python Programs

It is possible to integrate a part into the document, see 19.1. This is the most elegant method and is also preferable. Individual lines and line ranges can also be selected in the list of options. If a file is integrated, it is always as up-to-date as the document. It may also be useful to integrate program lines: `redLED = pyb.LED(1)`.

Attention: The integration of programs with the help of images is pointless.

Listing 19.1.: The program “Hello World” in Python for microcontroller boards is inserted from the file Blink.py.

```
%%
%% This is file ‘rename-to-empty-base.tex’,
%% generated with the docstrip utility.
%%
%% The original source files were:
%%
%% fileerr.dtx  (with options: ‘return’)
%%
%% This is a generated file.
%%
%% The source is maintained by the LaTeX Project team and bug
%% reports for it can be opened at https://latex-project.org/bugs/
%% (but please observe conditions on bug reports sent to that address!)
%%
%%
%% Copyright (C) 1993-2025
%% The LaTeX Project and any individual authors listed elsewhere
%% in this file.
%%
%% This file was generated from file(s) of the Standard LaTeX ‘Tools
     Bundle’.
%%
%%
%% It may be distributed and/or modified under the
%% conditions of the LaTeX Project Public License, either version 1.3c
%% of this license or (at your option) any later version.
%% The latest version of this license is in
%%    https://www.latex-project.org/lppl.txt
%% and version 1.3c or later is part of all distributions of LaTeX
%% version 2005/12/01 or later.
%%
%% This file may only be distributed together with a copy of the LaTeX
%% ‘Tools Bundle’. You may however distribute the LaTeX ‘Tools Bundle’
%% without such generated files.
%%
%% The list of all files belonging to the LaTeX ‘Tools Bundle’ is
%% given in the file ‘manifest.txt’.
%%
 \message{File ignored}
\endinput
%%
%% End of file ‘rename-to-empty-base.tex’.
```

../../Code/HelloWorld/Blink.py

20. Representation of Programs written for Arduino Boards

It is possible to integrate a part into the document, see 20.1. This is the most elegant method and is also preferable. Individual lines and line ranges can also be selected in the list of options. If a file is integrated, it is always as up-to-date as the document.

Attention: The integration of programs with the help of images is pointless.

Listing 20.1.: The program “Hello World” for an Arduino microcontroller boardis inserted from the file Blink.ino.

```
/**
*
*   @file Blink.ino
*
*   @brief Simple program for testing the configuration
*
*
*   Turns an LED on for one second, then off for one second, repeatedly.
*
*   Most Arduinos have an on-board LED you can control. On the UNO, MEGA and ZERO
*   it is attached to digital pin 13, on MKR1000 on pin 6. LED_BUILTIN is set to
*   the correct LED pin independent of which board is used.
*   If you want to know what pin the on-board LED is connected to on your Arduino
*   model, check the Technical Specs of your board at:
*
*   https://www.arduino.cc/en/Main/Products
*
*   modified 8 May 2014
*   by Scott Fitzgerald
*   modified 2 Sep 2016
*   by Arturo Guadalupi
*   modified 8 Sep 2016
*   by Colby Newman
*
*   This example code is in the public domain.
*
*   https://www.arduino.cc/en/Tutorial/BuiltInExamples/Blink
*/



/**
*   @brief the setup function runs once when you press reset or power the board
*
*   standard function of Arduino sketches
*
*   Initialization of the pin LED_BUILTIN as output
*
*   @param ---
*
*   @return void
*/
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

/**
*   @brief the loop function runs over and over again forever
*
*   standard function of Arduino sketches
*
*   swichting the led on / off for 1sec.
*
*   @param ---
*
*   @return void
*/

void loop() {
  digitalWrite(LED_BUILTIN, HIGH);   // turn the LED on (HIGH is the voltage level)
  delay(1000);                       // wait for a second
  digitalWrite(LED_BUILTIN, LOW);    // turn the LED off by making the voltage LOW
  delay(1000);                       // wait for a second
}
```

../../Code/Arduino/Blink/Blink.ino

21. Erstes Kapitel

...
Eine Speicherprogrammierbare Steuerung (SPS) ist ...

Eine Computerized Numerical Control (CNC) benötigt eine SPS (SPS) zur ...

22. CAGD

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.
Das Buch CAGD von Gerald Farin ist ein Klassiker über Splines. [Far02]

Die Norm 66025 zur Programmierung von CNC-Maschinen ist ebenfalls ein Klassiker; sie behandelt allerdings keine Splines. [DIN66025-2]

Herr F. Farouki hat sich sowohl mit der Programmierung von CNC-Maschinen als auch mit Splines beschäftigt. Sein Artikel[1] über einen Echtzeitinterpolator zeigt dies auch. [FS17]

Ein neue Dimension der Werkzeugmaschinen sind durch die Erfindung von 3D-Drucker entstanden. [Rus+07]

Ein anderer Aspekt der Automatisierungstechnik ist die Kommunikation. Durch die 5G-Technik ist hier ein weiterer Meilenstein erreicht worden.[2]

[1] Mitautor ist J. Srinathu
[2] Zaf+20.

A. Zeichnungen mit tikz

Das Paket tikz ist ein mächtiges Werkzeug zu erstellen von Grafiken. Es existieren vielen Einführungen. Hier werden nur die ersten Schritte gezeigt, so dass man leicht Flussdiagramme erzeugen kann.

Zeichnen einer Linie und Pfeile

B. Kriterien für eine gutes LaTeX-Projekt

Ein guter Bericht zeichnet sich nicht nur durch den guten Inhalt aus, sondern erfüllt auch formale Aspekte. Die folgende Liste soll helfen, grundlegende Regeln einzuhalten. Vor der Abgabe sollten alle Punkte geprüft und abgehakt werden.

- ☐ Wird eine geeignete Verzeichnisstruktur verwendet?
- ☐ Wird eine geeignete Aufteilung in Dateien verwendet?
- ☐ Werden aussagekräftige Verzeichnis- und Dateinamen verwendet?
  - ☐ Werden Verzeichnis- und Dateinamen wie z.B. report oder Hausarbeit vermieden?
  - ☐ Werden für Bilder aussagekräftige Namen und nicht „image1“verwendet?
  - ☐ Sind die Verzeichnis- und Dateinamen nicht zu lang?
  - ☐ Werden Sonderzeichen und Freizeichen vermieden?
- ☐ Werden Befehle wie \newline und \\ vermieden?
- ☐ Werden die LaTeX-Dateien übersichtlich gestaltet?
  - ☐ Werden in den LaTeX-Dateien Einrückungen verwendet?
  - ☐ Werden Freizeilen eingefügt?
  - ☐ Wird im Header der Dateien das Projekt erwähnt?
  - ☐ Wird im Header der Dateien die Hauptquellen erwähnt?
  - ☐ Wird im Header der Dateien der Autor erwähnt?
- ☐ Werden alle notwendigen Dateien abgegeben?
- ☐ Werden die temporären Dateien gelöscht?
- ☐ Werden Grafiken selber erstellt?
- ☐ Werden die Quellen der Bilder angegeben?
- ☐ Wird mit dem Kommando \cite zitiert?
- ☐ Wird eine bib-Datei verwendet?
- ☐ Sind die Einträge der bib-Datei übersichtlich?
- ☐ Sind die Einträge der bib-Datei so editiert, dass sie korrekt dargestellt werden?
- ☐ Werden die korrekten Typen für die bib-Einträge verwendet?
- ☐ Werden aussagekräftige Schlüssel in den bib-Dateien verwendet?

Leider kann die Liste nicht vollständig sein, aber sie liefert einige Anhaltspunkte.

Literaturverzeichnis

[Aba+16] M. Abadi u. a. "TensorFlow: A System for Large-Scale Machine Learning". In: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. Savannah, GA: USENIX Association, Nov. 2016, S. 265–283. ISBN: 978-1-931971-33-1. URL: https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi.

[DIN66025-2] DIN Deutsches Institut für Normung e.V. *DIN 66025-2:1988-09: Programmaufbau für numerisch gesteuerte Arbeitsmaschinen: Wegbedingungen und Zusatzfunktionen*. Norm. Berlin, Sep. 1988.

[Eck23] T. Eckel. *NewPing Library Documentation*. [pdf]. 2023. URL: playground.arduino.cc/Code/NewPing.

[Ele20] S. Electronics. *Ultrasonic Ranging Module HC - SR04*. Accessed: 2024. 2020. URL: https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf.

[FS17] R. Farouki und J. Srinathu. "A real-time CNC interpolator algorithm for trimming and filling planar offset curves". In: *Computer-Aided Design* 86 (Jan. 2017). DOI: 10.1016/j.cad.2017.01.001.

[Far02] G. Farin. *Curves and Surfaces for CAGD*. 5. [pdf]. San Diego, CA: Academic Press, 2002.

[Fra16] J. Fraden. *Handbook of Modern Sensors: Physics, Designs, and Applications*. Springer, 2016.

[Jak+10] G. Jaklic u. a. "On Interpolation by Planar Cubic G^2 Pythagorean-Hodograph Spline Curves". In: *Mathematics of Computation* (2010). [pdf].

[KT-12] KT-elektronic. *Ultraschall Messmodul HC-SR04 Beschreibung*. 2012. URL: https://www.mikrocontroller.net/attachment/218122/HC-SR04_ultraschallmodul_beschreibung_3.pdf.

[Raj12] S. Raj. *Towerpro SG90 Servo Motor And Its Application*. Geöffnet: 16.04.2026. 3.12.2022. URL: https://vayuyaan.com/blog/towerpro-sg90-servo-motor-and-its-application/.

[Rus+07] D. Russell u. a. *Apparatus and Methods for 3D Printing*. United States Patent, Patent N0.: US 7,291,002 B2. 2007.

[Wat17a] Waterloo Maple Inc. 2017. *Description*. letzter Zugriff 14.09.2017. 2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/description.

[Wat17b] Waterloo Maple Inc. 2017. *Export*. letzter Zugriff 14.09.2017. 2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/export.

[Wat17c] Waterloo Maple Inc. 2017. *Module*. letzter Zugriff 14.09.2017. 2017. URL: https://de.maplesoft.com/support/help/maple/view.aspx?path=module.

[Wat17d] Waterloo Maple Inc. 2017. *ModuleLoad*. letzter Zugriff 14.09.2017. 2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=ModuleLoad.

[Wat17e] Waterloo Maple Inc. 2017. *Option*. letzter Zugriff 14.09.2017. 2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/option.

[Wat17f] Waterloo Maple Inc. 2017. *Procedures*. letzter Zugriff 14.09.2017. 2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=procedure.

[Zaf+20] A. Zafeiropoulos u. a. "Benchmarking and Profiling 5G Verticals' Applications: An Industrial IoT Use Case". In: *IEEE Conference on Network Softwarization, NetSoft 2020*. 2020. DOI: 10.1109/NetSoft48620.2020.9165393.

Index

3D-Drucker, 105
5G, 105

Speicherprogrammierbare Steuerung, 103
Datei
    Blink.ino, 102
    Blink.py, 100
    PackageExample.py, 67
    requirements.txt, 65

CAGD, 105
    Splines, 105
CNN, 11, 103
Computerized Numerical Control
    siehe CNN, 11, 103

Example, 67
    Description, 67
    Installation, 67
    Introduction, 67

Speicherprogrammierbare Steuerung
    siehe SPS, 11, 103
SPS, 11, 103, *siehe* Speicherprogrammierbare Steuerung